

MARTIN-LUTHER-UNIVERSITÄT
HALLE-WITTENBERG

Forschungsgruppenmodul: Advanced Bioinformatics

Optimierung der Assemblierung des hochrepetitiven TALoms in Xanthomonas oryzae pv. oryzae (PXO35) durch vergleichende Analyse von Long-Read-, Short-Read- und Hybrid-Sequenzierungsansätzen sowie deren Annotation mittels AnnoTALE

Betreuer:	Dr. rer. nat. Jan Grau
vorgelegt von:	Ahmad Wattar
	217234191
Studiengang:	M.Sc. Bioinformatik
Abgabetermin:	7. August 2025

Inhaltsverzeichnis

1 Einleitung **3**

1.1 Hintergrund zu *Xanthomonas* . . . 3

1.2 Ziel der Arbeit . . . 4

2 Grundlagen **5**

2.1 Sequenziermethoden . . . 5

2.2 Genomassemblierung . . . 5

2.3 Polierung . . . 6

2.3.1 Computergestützte Korrektur von TALEs in ONT-Assemblies . . . 6

2.4 Annotation . . . 6

3 Methodik und Durchführung **7**

3.1 Datengrundlage . . . 7

3.2 Daten Vorbereitung: . . . 7

3.3 Scaffolding der Illumina-basierten SPAdes-Assemblierung mit ONT-Reads unter Verwendung von RagTag und anschließende Politur . . . 9

3.4 Assemblierung der Illumina-Daten mit SPAdes, gefolgt von einer Politur der Assembly mit Medaka unter Einsatz von Rohdaten und gefilterten ONT-Reads . . . 9

3.5 Politur der Illumina-SPAdes-Assemblierung mit Pilon . . . 9

3.6 ONT-basierte Assemblierung mit Flye (mit Filterung auf 500 Mb), Politur mit Medaka und Pilon . . . 10

3.7 ONT-basierte Assemblierung mit Flye (mit Filterung auf 350 Mb), Politur mit Medaka und Pilon . . . 11

3.8 ONT-basierte Assemblierung mit Flye (mit Filterung auf 500 Mb, 2 Iterationen), Politur mit Racon, Medaka und abschließende Pilon-Politur . . . 12

3.9 Flye-Assembly mit zwei Iterationen, Politur mit Racon und Medaka unter Verwendung gefilterter ONT-Daten und Vergleich verschiedener Mapping-Parameter . . . 13

3.10 Flye-Assemblierung ohne interne Politur, anschließend verfeinert mit Racon, Medaka und Pilon . . . 14

3.11 ONT-Assembly mit Polieren durch PacBio-Daten – technische Herausforderungen bei der Tool-Installation und Datenkonvertierung . . . 16

3.12 ONT-Assemblierung mit Raven und anschließender Politur mit Racon, Medaka und Pilon . . . 17

3.13 ONT-Assemblierung mit Canu und Politur mit Racon, Medaka und Pilon . . . 18

3.14 Hybrid-Assemblierung mit Unicycler unter Nutzung von Illumina- und ONT-Daten . . . 21

3.15 Politur mit Homopolish auf Basis der Flye-Assemblierung aus Abschnitt 3.6 und abschließender Pilon-Polierung . . . 22

3.16 Hybridassemblierung mit MaSuRCA . . . 23
3.17 Illumina-basierte Assemblierung mit MaSuRCA . . . 23
3.18 Polierung der mit Flye assemblierten ONT-Daten mittels Computational Correction . . 23
3.19 Annotation mit AnnoTALE . . . 24
3.20 Visualisierung mit ProgressiveMauve . . . 26
3.21 Pseudogene Alignments . . . 29
3.22 Analyse des einzigen TALE-Pseudogens nach comp. correction . . . 29

4 Disskusion **30**
4.1 Persönlicher Eindruck . . . 31

5 Literatur **32**

Abbildungsverzeichnis **35**

Abkürzungsverzeichnis **37**

A Alignments der TALE-Proteinsequenzen **38**

B T-Coffee Alignment-Ergebnis **41**

1 Einleitung

1.1 Hintergrund zu *Xanthomonas*

Xanthomonas-Bakterien zählen zu den wichtigsten Erregern von Pflanzenkrankheiten. Sie befallen zahlreiche landwirtschaftlich genutzte Kulturen wie Reis, Baumwolle, Kohl, Zitrusfrüchte und Gerste und verursachen weltweit erhebliche Ertragseinbußen. Besonders schwerwiegend ist ihr Einfluss im Reisanbau, wo Infektionen durch Xanthomonas zu bakteriellen Reiserkrankungen führen können, die Ernteverluste von bis zu 50 Prozent verursachen könnten. Im Verlauf der Infektion schleusen die Bakterien verschiedene Effektorproteine direkt in die Zellen ihrer Wirtspflanzen ein. Diese greifen gezielt in pflanzliche Zellprozesse ein und fördern sowohl die Krankheitsentwicklung als auch die Verbreitung der Erreger im Gewebe. Eine besonders wichtige Untergruppe dieser Effektorproteine sind die Transcription Activator-Like Effectors (TALEs). TALEs agieren als transkriptionelle Aktivatoren von Genen der Wirtspflanze. Ein einzelner Xanthomonas-Stamm kann dabei bis zu 28 verschiedene TALEs besitzen (Grau et al., 2024; Mücke et al., 2019). Der strukturelle Aufbau einer TALE ist durch eine charakteristische DNA-Bindungsdomäne geprägt, die aus einer Vielzahl nahezu identischer Tandem-Wiederholungen besteht. Jede dieser Wiederholungseinheiten umfasst 34 konservierte Aminosäuren und bildet gemeinsam eine rechtsgängige Superhelix. Diese Helixstruktur windet sich entlang der DNA und ermöglicht eine sequenzspezifische Erkennung und Bindung an Promotorregionen pflanzlicher Zielgene. Innerhalb dieser Wiederholungen sind vor allem die Aminosäuren an Position 12 und 13 von besonderer Bedeutung. Diese Repeat Variable Di-Residues (RVDs) bestimmen maßgeblich die Zielsequenzspezifität der TALE. Jeder RVD ist in der Lage, ein spezifisches Nukleotid der DNA zu erkennen und zu binden. Während der überwiegende Teil der Aminosäuren innerhalb der Wiederholungen stark konserviert ist, zeigen die RVD-Positionen eine hohe Variabilität. Diese Variabilität ist entscheidend für die Anpassungsfähigkeit von TALEs an unterschiedliche DNA-Sequenzen ihrer Wirtsorganismen (Erkes et al., 2017). Die Analyse und Assemblierung des TALoms, der Gesamtheit aller TALE-Gene innerhalb eines Xanthomonas-Stamms, ist entscheidend, um evolutionäre Prozesse in Xanthomonas-Populationen zu verstehen. Sie liefert wichtige Erkenntnisse über die Häufigkeit, Bedeutung und Vielfalt bestimmter TALE-Klassen sowie über die Mechanismen, die zur Umlagerung und Weiterentwicklung dieser Effektorproteine führen. Eine zentrale Herausforderung bei der Entschlüsselung von TALomen liegt in der starken Wiederholungsstruktur der TALE-Gene. Die DNA-Bindungsdomäne besteht aus nahezu identischen, konservierten Wiederholungseinheiten, und TALE-Gene treten häufig in Gruppen im Genom auf. Diese hohe Repetitivität erschwert eine zuverlässige Assemblierung, insbesondere mit Sequenzierungstechnologien, die auf kurzen Leselängen basieren, wie der Illumina-Technologie. In vielen Fällen werden TALE-Gene bei solchen Analysen unvollständig erfasst oder ganz ausgelassen. Long-Read-Technologien wie PacBio oder Oxford Nanopore sind daher unverzichtbar, da sie diese repetitiven Regionen deutlich besser assemblieren können. Da ONT-Reads jedoch ein spezifisches Fehlerprofil aufweisen, ist eine gezielte rechnerische Korrektur notwendig, um diese Mängel auszugleichen und so eine vollständige und korrekte Erfassung der TALome sicherzustellen (Erkes et al., 2023).

1.2 Ziel der Arbeit

Ziel dieser Arbeit ist es, die Auswirkungen verschiedener Sequenziermethoden auf die Qualität der Assemblierung eines hochrepetitiven bakteriellen Genoms am Beispiel eines Xanthomonas-Stamms zu untersuchen. Im Fokus stehen die sogenannten TALome-Regionen, die aufgrund ihrer repetitiven Struktur besondere Anforderungen an die Assemblierung stellen. Es soll analysiert werden, wie sich unterschiedliche Sequenziertechnologien, Qualitätskontrollen, Filterstrategien, Assemblierungsalgorithmen und Polierungsmethoden auf die Vollständigkeit und Genauigkeit der Genomrekonstruktion auswirken. Dabei werden verschiedene Werkzeuge und Parameterkombinationen getestet, um optimale Bedingungen für eine möglichst präzise Assemblierung zu identifizieren. Zur Bewertung der Assemblierungsqualität wird das Tool *AnnoTALE* eingesetzt, das eine spezifische Annotation von TALE-Genen ermöglicht. Ziel ist es zu ermitteln, ob die Assemblierung funktionale TALE-Gene korrekt abbildet oder lediglich unvollständige bzw. pseudogene Sequenzen erzeugt. Insgesamt soll die Arbeit zeigen, unter welchen methodischen Bedingungen eine zuverlässige Rekonstruktion des TALomes gelingt.
Die Arbeit bezieht sich dabei auf die Publikation *"Assembling highly repetitive Xanthomonas TALomes using Oxford Nanopore sequencing"* (`https://doi.org/10.1186/s12864-023-09228-1`), in der ein methodischer Rahmen zur erfolgreichen Assemblierung repetitiver TALome-Regionen unter Verwendung von Oxford Nanopore-Sequenzdaten vorgestellt wird.

2 Grundlagen

2.1 Sequenziermethoden

Im Gegensatz zur Sanger-Sequenzierung, die enorme Kosten erforderte, um das erste menschliche Genom zu entschlüsseln, ermöglichten die entwickelten Next-Generation-Sequencing-Technologien (NGS) eine massive Parallelisierung von Sequenzierungsreaktionen. NGS-Verfahren lassen sich grob in zwei Kategorien unterteilen: Short-Read- und Long-Read-Sequenzierung. Short-Read-Technologien wie Illumina basieren auf der parallelen Sequenzierung kurzer, amplifizierter DNA-Fragmente mit typischen Längen von 250 bis 800 Basenpaaren. Die Methode liefert äußerst genaue Daten, stößt jedoch bei der Assemblierung komplexer oder repetitiver Genomregionen an ihre Grenzen. Die Long-Read-Sequenzierung überwindet diese Limitierung, indem sie sehr lange DNA-Moleküle direkt liest, meist mit einer Länge von über 10 Kilobasen. Eine zentrale Technologie der Long-Read-Sequenzierung ist Oxford Nanopore Technologies (ONT). Dabei wird ein einzelner DNA- oder RNA-Strang durch eine Nanopore geleitet, wobei charakteristische Veränderungen im Ionenstrom auftreten. Diese Stromsignale werden in Echtzeit erfasst und ermöglichen die Bestimmung der Basenabfolge. ONT zeichnet sich durch lange Reads und die Fähigkeit zur direkten Erkennung epigenetischer Modifikationen aus, weist jedoch im Vergleich zu Short-Read-Technologien eine höhere Fehlerrate auf, die durch bioinformatische Korrekturverfahren reduziert werden kann (Hu et al., 2025).

2.2 Genomassemblierung

Die Assemblierung ist der Prozess, bei dem Sequenzfragmente (Reads), die bei der DNA-Sequenzierung entstehen, zu längeren, zusammenhängenden Abschnitten (Contigs) und schließlich zu vollständigen Genomsequenzen zusammengesetzt werden. Eine früh verwendete Methode war der Overlap-Layout-Consensus-Ansatz. Dabei wurden zuerst überlappende Bereiche zwischen den Reads erkannt. Danach wurden die besten Überlappungen nacheinander zusammengefügt, wodurch Contigs entstanden. Dieser Schritt ähnelt einem multiplen Sequenzalignment, bei dem auch Lücken und Fehler (Mismatches) auftreten können. Abschließend wurde aus dem Alignment eine Konsensussequenz erzeugt, die die wahrscheinlichste Sequenzvariante basierend auf den überlappenden Reads repräsentiert. Problematisch waren jedoch repetitive Elemente im Genom, vor allem wenn sie länger waren als die typische Leseweite. Dann ließ sich ihre genaue Position oft nicht eindeutig bestimmen. Da diese Methode bei großen Datenmengen sehr aufwendig ist und moderne Sequenziertechnologien heute Millionen Reads liefern, wurde ein neuer Ansatz notwendig. Dieser basiert auf der Zerlegung der Reads in kurze Abschnitte (k-mere) und der Darstellung als de Bruijn-Graph. In diesem Graphen stehen Knoten für k-mere, und Kanten zeigen an, wenn sich zwei k-mere überlappen. Die gesuchte Sequenz entspricht einem Pfad im de Bruijn-Graphen, der jede Kante exakt einmal durchläuft, einem Eulerkreis. Dadurch kann das Genom auch bei großen Datenmengen effizient rekonstruiert werden.(Merkl, 2022).

2.3 Polierung

Die Polierung stellt einen entscheidenden Schritt in der Genomassemblierung dar, bei dem ein zunächst zusammengesetztes Genom durch Nutzung der zugrundeliegenden Sequenzdaten verbessert wird. Ziel dieses Prozesses ist die Korrektur von Fehlern, die infolge der begrenzten Genauigkeit der Sequenziertechnologien entstehen. Dies betrifft insbesondere Methoden wie Oxford Nanopore oder PacBio, welche lange Reads erzeugen, jedoch mit einer höheren Fehlerrate verbunden sein können. Durch die Polierung entsteht eine präzisere, fehlerärmere und biologisch verlässlichere Genomrepräsentation. Dabei werden die ursprünglichen Sequenzreads erneut gegen das zusammengesetzte Genom ausgerichtet (Mapping), um Abweichungen zwischen den Reads und der Assemblierung zu erkennen und zu korrigieren. Die Polierung kann Einzelbasenfehler, kleine Einfügungen und Löschungen (Indels), größere strukturelle Fehler sowie falsch zusammengesetzte Bereiche beheben (Sovic, 2017), (Oxford Nanopore Technologies, 2019), (Walker and Langhorst, 2015).

2.3.1 Computergestützte Korrektur von TALEs in ONT-Assemblies

Die ONT-Technologie ermöglicht durch lange Reads die Erfassung kompletter TALE-Cluster, ist jedoch fehleranfällig, insbesondere in Homopolymer-Regionen und bei geringer Abdeckung. Daher wird das Tool *Talecorrect*eingesetzt, um gezielt Insertions und Deletions in TALE-Regionen zu korrigieren, indem es drei separat trainierte Hidden-Markov-Modelle (HMMs) verwendet. Diese Modelle basieren auf bekannten TALE-Sequenzen aus AnnoTALE und werden mithilfe von nhmmer auf die ONT-Assemblies angewendet. Nur Treffer mit hoher Qualität werden berücksichtigt. Innerhalb der erkannten TALE-Abschnitte identifiziert das Tool kurze, zusammenhängende Einfügungen oder Löschungen und korrigiert diese nach festgelegten Regeln. Eine Besonderheit dabei ist, dass der hochvariable RVD-Bereich in den Repeats bewusst unverändert bleibt, um Fehlkorrekturen zu vermeiden. (Erkes et al., 2023).

2.4 Annotation

Annotation ist der Vorgang, bei dem biologische Informationen einer bereits bekannten DNA-Sequenz zugeordnet werden. Dabei werden mit Hilfe bioinformatischer Werkzeuge funktionelle Einheiten wie Gene erkannt. Im nächsten Schritt erfolgt die Zuordnung konkreter Funktionen zu diesen Abschnitten, häufig durch den Vergleich mit bekannten Genen in Datenbanken. Das Ziel der Annotation besteht darin, aus der reinen DNA-Sequenz ein Genom mit funktioneller Bedeutung zu erstellen (Merkl, 2022). Für diese Arbeit wird das Tool *AnnoTALE* verwendet, um gezielt TALE-Sequenzen zu annotieren.

3 Methodik und Durchführung

3.1 Datengrundlage

Alle in dieser Studie verwendeten Sequenzierdaten sind öffentlich über das European Nucleotide Archive (ENA) unter der Projektzugangsnummer `PRJEB54678` verfügbar. Diese Daten dienten als Grundlage für sämtliche in dieser Arbeit durchgeführten Analysen.

3.2 Daten Vorbereitung:

Zur Qualitätsoptimierung der ONT- und Illumina-Reads wurden verschiedene *Filtlong*-Parameter (Version v0.2.1) getestet und deren Auswirkungen auf die Read-Qualität untersucht. Tabelle 1 zeigt die Ergebnisse dieser Strategien im Hinblick auf Read-Länge, N50-Wert und Qualitätsverteilung. Die erste Zeile der Tabelle stellt die ONT-Rohdaten ohne Filterung dar. Dabei fällt auf, dass eine gezielte Reduktion der Gesamtdatenmenge, beispielsweise mit `--target_bases 500M`, die durchschnittliche Read-Länge (mean read length) und den N50-Wert deutlich erhöht, während gleichzeitig der Anteil der Reads mit einer Qualität über Q10 ansteigt. Die Einstellung `--target_bases 350` liefert sogar eine noch höhere mittlere Read-Länge, den besten N50-Wert sowie nahezu 100 % Reads über Q10. Allerdings geht dies mit einer reduzierten Gesamtanzahl an Reads einher, was potenziell nachteilig für die Assemblierung sein kann. Hier muss also ein sinnvoller Kompromiss zwischen Read-Qualität und -Menge gefunden werden. Bei der Verwendung von Illumina-Reads als Referenz mit `Filtlong` (z. B. `--target_bases 500M` plus Illumina) werden ONT-Reads anhand eines k-mer-basierten Scorings gefiltert. Dabei werden bevorzugt Reads ausgewählt, die gut zu den Illumina-Daten passen. Dies führt typischerweise zu einer höheren Basenqualität, jedoch sind die resultierenden Reads im Durchschnitt kürzer und haben einen niedrigeren N50-Wert als ohne Illumina-Referenz. Im Gegensatz dazu führt die Option `--trim --split` zu stark fragmentierten Reads und ist daher für die Assemblierung ungeeignet. Alternativ zu `--target_bases` kann auch der Parameter `--keep_percent` verwendet werden. Dieser arbeitet prozentbasiert statt mit einer absoluten Zielabdeckung und führt zu einem vergleichbaren Ergebnis. Basierend auf diesen Ergebnissen wurden beide Strategien – 500M und 350M – für die weiterführenden Analysen ausgewählt.
Die Illumina-Daten wurden vor der Analyse mit *fastp* (Version v0.24.3) unter Verwendung der Standardparameter gefiltert, um die Datenqualität zu verbessern. Tabelle 2 zeigt die Qualitätsmetriken vor und nach der Filterung. Nach dem Filterprozess ist eine Steigerung des Anteils der Basen mit Q20- und Q30-Qualität erkennbar, was auf eine verbesserte Datenqualität hinweist, während die Gesamtzahl der Reads und Basen nur leicht reduziert wurde. Der GC-Gehalt blieb stabil, was auf eine unveränderte Zusammensetzung der Daten hindeutet. Insgesamt ermöglicht die sorgfältige Filterung der Rohdaten mit *filtlong* und *fastp* eine verbesserte Grundlage für nachfolgende Assemblierungs- und Politur-Schritte und stellt einen sehr wichtigen Anfangsschritt dar.

Tabelle 1: Auswirkung verschiedener `Filtlong`-Strategien auf ONT-Read-Qualität

Strategie	**Illumina**	**Basen (Mb)**	**m.r.length (Ø)**	**N50 (bp)**	**% Q>10**
Original	nein	3310.5	5708	23888	74.6
`--target_bases 500M`	nein	500.0	46023	48339	99.5
`--target_bases 463M`	nein	463.5	46714	48958	91.2
`--target_bases 350M`	nein	350.0	48992	51041	99.9
`--keep_percent 15.1`	nein	495.0	43000	47100	97.8
`--keep_percent 14.5`	nein	480.0	46425	48708	99.6
`--target_bases 500M`	ja	500.0	33772	39500	98.5
`--target_bases 1G`	nein	1000.0	39678	43656	96.7
`--target_bases 750M`	nein	750.0	42185	45341	98.3
`--target_bases 600M`	nein	600.0	44305	46906	99.1
`--trim --split`	–	–	–	–	–

Tabelle 2: Read-Qualität bei Illumina

Metric	**Before Filtering**	**After Filtering**
Total Reads (M)	5.956394	5.395032
Total Bases (M)	595.639400	539.103086
Q20 Bases (M)	535.294310	503.309694
Q20 Percentage (%)	89.868855	93.360566
Q30 Bases (M)	465.523001	441.064656
Q30 Percentage (%)	78.155173	81.814530
GC Content (%)	63.415247	63.286071

3.3 Scaffolding der Illumina-basierten SPAdes-Assemblierung mit ONT-Reads unter Verwendung von RagTag und anschließende Politur

Zunächst wurden Illumina-Daten mittels *SPAdes* (Version 4.2.0) assembliert, um Contigs zu generieren. Anschließend kamen ONT-Reads in Kombination mit dem Tool *RagTag* (Version 2.1.0) zum Einsatz, um die Illumina-Contigs zu scaffolden. Dies führte zu größeren, zusammenhängenden Scaffolds. Das Scaffolding resultierte jedoch in insgesamt 327.228 Scaffolds aus lediglich 3.642 ursprünglichen Contigs, was auf ein unzureichendes Scaffolding hindeutet. Bei der anschließenden Politur mit *Medaka* (Version 2.0.1) traten Schwierigkeiten auf, da die Scaffold-Header nicht dem erforderlichen Format entsprachen, wodurch die Politur verhindert wurde. Mehrere Versuche, die Scaffold-Datei anzupassen, um sie für die Politur mit *Medaka* nutzbar zu machen, blieben erfolglos. Ziel des Vorgehens war es, auf der Grundlage der ursprünglich erzeugten Illumina-Assemblierung mit hoher Basengenauigkeit durch Scaffolding mit ONT-Reads eine verbesserte Gesamtkontigierung zu erreichen. Im Anschluss sollte durch die Politur eine Reduktion der Scaffold-Anzahl erzielt werden, um eine möglichst zusammenhängende Assemblierung zu erhalten. Die hohe Anzahl an resultierenden Scaffolds aus nur 3.642 Contigs deutet jedoch darauf hin, dass dieses Ziel nicht erreicht wurde.

3.4 Assemblierung der Illumina-Daten mit SPAdes, gefolgt von einer Politur der Assembly mit Medaka unter Einsatz von Rohdaten und gefilterten ONT-Reads

Zunächst wurde eine Assemblierung der Illumina-Daten mit *SPAdes* durchgeführt. Diese Assemblierung zeigte sehr gute Ergebnisse mit BUSCO-Vollständigkeiten (Version 5.8.3) von 98,9 % (Xanthomonadales-Datenbank) und 97,4 % (Bacteria-Datenbank). Anschließend wurde versucht, die Assemblierung mithilfe von *Medaka* unter Verwendung von Rohdaten aus ONT zu polieren. Die daraus resultierende Politur wies jedoch eine geringere BUSCO-Vollständigkeit von 95,1 % auf. Daraufhin wurde *Medaka* erneut ausgeführt, diesmal unter Verwendung gefilterter ONT-Reads. Das Ergebnis war jedoch nochmals schlechter, mit einer BUSCO-Vollständigkeit von lediglich 88,9 %. Diese Ergebnisse deuten darauf hin, dass die Politur mit ONT-Rohdaten oder gefilterten ONT-Daten die Qualität der ursprünglichen *Illumina-SPAdes*-Assemblierung eher verschlechtert als verbessert hat.

3.5 Politur der Illumina-SPAdes-Assemblierung mit Pilon

Basierend auf der hochwertigen *SPAdes*-Assemblierung aus Abschnitt 3.4 wurde eine Politur mit *Pilon* (Version 1.24) unter Verwendung der *Illumina*-Reads durchgeführt. Dazu wurde die Assemblierung zunächst indexiert, die Reads mithilfe von *BWA* (Version 0.7.19, r1273) gemappt und anschließend mit *Samtools* sortiert sowie indexiert. Die Politur mit *Pilon* erfolgte unter Verwendung der Standardparameter. Die BUSCO-Analyse zeigte eine unveränderte Vollständigkeit, sodass durch die *Pilon*-Politur keine Qualitätsverbesserung erzielt wurde.

3.6 ONT-basierte Assemblierung mit Flye (mit Filterung auf 500 Mb), Politur mit Medaka und Pilon

Für die Genomassemblierung wurden ONT-Daten verwendet, die zuvor mittels *Filtlong* auf eine Zielabdeckung von 500 Megabasen gefiltert wurden. Diese gefilterten Reads wurden anschließend mit *Flye* (Version 2.9.5) assembliert, wobei nur eine Iteration unter Verwendung der Parameter `--meta` und `--min-overlap 500` durchgeführt wurde. Die resultierende Assemblierung erreichte in der BUSCO-Analyse 82,8 Prozent vollständige Gene mit der Datenbank *xanthomonadales_odb10* und 88,8 Prozent mit der allgemeinen *bacteria_odb10*-Datenbank. Im nächsten Schritt wurde die Assemblierung mit *Medaka* unter Verwendung der ONT-Rohdaten poliert. Diese Polierung führte jedoch zu keiner Verbesserung der BUSCO-Werte: Für Xanthomonadales sank der Wert geringfügig auf 82,6 Prozent, während der Wert für Bacteria unverändert bei 88,8 Prozent blieb. Auch weitere Durchläufe mit Medaka, zwei beziehungsweise drei Runden, konnten die Ergebnisse nicht verbessern. Zur weiteren Qualitätssteigerung wurde eine Polierung mit *Pilon* durchgeführt. Dafür wurden die zuvor gefilterten Illumina-Daten verwendet. Die Reads wurden mit *BWA* auf die Medaka-polierte Assemblierung gemappt und anschließend mit Pilon poliert. Zwei Varianten der Polierung wurden getestet: einmal mit den Parametern `--fix bases --fix indels` und einmal mit `--fix all`. Die erste Variante erreichte in der BUSCO-Analyse 98,5 Prozent für Xanthomonadales und 97,4 Prozent für Bacteria, während die zweite Variante 98,8 Prozent für Xanthomonadales und ebenfalls 97,4 Prozent für Bacteria erreichte. Eine zweite Runde mit Pilon konnte den BUSCO-Wert mit der Xanthomonadales-Datenbank noch leicht auf 98,9 Prozent steigern.Insgesamt zeigt sich, dass die initiale Flye-Assemblierung mit den ONT-Daten eine solide Grundlage liefert, jedoch erst durch die Polierung mit Illumina-Daten signifikant verbessert werden kann. Medaka alleine bringt keinen spürbaren Zugewinn an Vollständigkeit, während Pilon insbesondere mit der Einstellung `--fix all` die Assemblierungsqualität erhöht.
Es wurden außerdem verschiedene Assemblierungsversuche mit erhöhten Overlap-Werten, beispielsweise 3000 bp, durchgeführt. Diese konnten jedoch aufgrund unzureichender RAM-Ressourcen nicht erfolgreich abgeschlossen werden.

3.7 ONT-basierte Assemblierung mit Flye (mit Filterung auf 350 Mb), Politur mit Medaka und Pilon

Ein weiterer Versuch wurde durchgeführt, bei dem die Filtlong-Filterung auf eine Zielabdeckung von 350 Millionen Basen eingestellt wurde, im Gegensatz zu den zuvor verwendeten 500 Millionen. Die so gefilterten ONT-Daten wurden anschließend erneut mit *Flye* assembliert, wobei dieselben Parameter wie im vorherigen Versuch verwendet wurden, nämlich `--meta` und `--min-overlap 5000`. Die resultierende Assemblierung erzielte in der BUSCO-Analyse mit der *xanthomonadales_odb10*-Datenbank einen Wert von 84,6 Prozent vollständiger Gene und lag damit etwas höher als im ersten Versuch, bei dem 82,8 Prozent erreicht wurden. Im Anschluss wurde auch diese Assemblierung mit *Medaka* unter Verwendung der ONT-Rohdaten poliert. Die Polierung führte jedoch zu keiner weiteren Verbesserung: Der BUSCO-Wert blieb mit 84,6 Prozent unverändert. Somit zeigt sich, dass die geringere Filtlong-Zielabdeckung von 350 Millionen Basen bereits eine etwas vollständigere Assemblierung ermöglicht, Medaka in diesem Fall jedoch keinen zusätzlichen Gewinn brachte. Anschließend wurde wie zuvor eine Polierung mit *Pilon* durchgeführt, wobei die Option `--fix all` verwendet wurde. Bereits nach einer Runde konnte ein BUSCO-Wert von 98,9 Prozent mit der *xanthomonadales_odb10*-Datenbank erzielt werden, was den hohen Nutzen der Illumina-basierten Polierung auch in diesem Fall bestätigt.

Die folgende Tabelle 3 zeigt einen Vergleich der Ergebnisse aus den Abschnitten 3.6 und 3.7 Dabei werden die Daten der ONT-Sequenzierung aus Abschnitt 3.6 mit einer Länge von 500 Mb den Ergebnissen aus Abschnitt 3.7 mit einer Länge von 350 Mb gegenübergestellt.

Tabelle 3: Vergleich der Ergebnisse mit verschiedenen Coverage, Busco nach xanthomonadales

Analyse Schritt	**ONT (500 Mb)**	**ONT (350 Mb)**
Filtlong	500 M	350 M
Flye Assembly	`--iterations 1`	`--iterations 1`
BUSCO nach Flye	82.8 %	84,6 %
Medaka	ja	ja
BUSCO nach Medaka	82,6	84,6
Pilon	ja	ja
BUSCO nach Pilon	98,8	98,9
2. Runde Pilon	ja	Nein
BUSCO nach Pilon	98,8	-

3.8 ONT-basierte Assemblierung mit Flye (mit Filterung auf 500 Mb, 2 Iterationen), Politur mit Racon, Medaka und abschließende Pilon-Politur

In einem weiteren Versuch wurden die 500 Millionen ONT-Daten mit *Flye* assembliert, wobei erneut die Parameter `-meta` und `-min-overlap 5000` verwendet wurden. Im Unterschied zur vorherigen Assemblierung wurde diesmal die Option `-iterations 2` anstelle von nur einer Iteration genutzt. Dadurch konnte ein leicht verbessertes BUSCO-Ergebnis erzielt werden: 83,0 % für das *xanthomonadales_odb10* und 88 % für das *bacteria_odb10*. Anschließend erfolgte eine Polierung mit *Racon* (Version 1.4.3) unter Verwendung der Rohdaten der ONT-Sequenzierung, wobei das Mapping mit *Minimap2* (Version 2.29) durchgeführt wurde. Nach dieser Polierung sank der BUSCO-Wert jedoch auf 64,4 % für das *xanthomonadales_odb10*. Durch eine anschließende Medaka-Polierung konnte dieser Wert auf 72,9 % verbessert werden. Eine nachfolgende Polierung mit Pilon unter Verwendung der Illumina-Daten erhöhte den BUSCO-Wert schließlich auf 98,2 %. Sowohl bei *Racon* als auch bei *Medaka* wurden die Rohdaten der ONT-Sequenzierung zur Polierung verwendet, was möglicherweise die Ursache für die zunächst geringeren BUSCO-Werte sein könnte.

In Tabelle 4 sind die Ergebnisse eines Vergleichs zwischen Abschnitt 3.6 und Abschnitt 3.8 dargestellt. Dabei wurde im Abschnitt 3.6 die Flye-Assembly mit nur einer Iteration der Politur durchgeführt, während im Abschnitt 3.8 die Flye-Assembly mit zwei Iterationen poliert wurde.

Tabelle 4: Vergleich der Ergebnisse in Bezug auf die Anzahl der Polierungsiterationen bei Flye und Racon

Analyse-Schritt	**ONT (500 Mb)**	**ONT (500 Mb)**
Flye Assembly	`--iterations 1`	`--iterations 2`
BUSCO (Xanthomonadales, vor Racon)	82,8 %	83,0 %
Racon Polishing	nicht durchgeführt	1 Runde durchgeführt
BUSCO (Xanthomonadales, nach Racon)	-	64,4 %
Medaka Polishing	durchgeführt	durchgeführt
BUSCO nach Medaka	82,6 %	72,9 %
Pilon Polishing	durchgeführt	durchgeführt
BUSCO nach Pilon	98,8 %	98,2 %

3.9 Flye-Assembly mit zwei Iterationen, Politur mit Racon und Medaka unter Verwendung gefilterter ONT-Daten und Vergleich verschiedener Mapping-Parameter

In einem weiteren Versuch wurde das Flye-Assembly aus dem vorherigen Versuch verwendet, das mit zwei Iterationen erstellt worden war. Für die anschließende Polierung mit *Racon* kamen diesmal aber die gefilterten ONT-Daten zum Einsatz. Das Mapping erfolgte zunächst mit *Minimap2* unter Standardparametern. Die darauffolgende Politur mit *Racon* führte zu einem BUSCO-Wert von 78,5 % für das *bacteria* und 74,9 % für das *xanthomonadales_odb10*. Insgesamt zeigte sich damit erneut eine Verschlechterung im Vergleich zum ursprünglichen Flye-Ergebnis. Dennoch fällt der BUSCO-Wert in diesem Versuch höher aus als im vorherigen, bei dem *Racon* mit den Rohdaten durchgeführt wurde. Daraufhin wurde ein alternativer Mapping-Ansatz mit strengeren Parametern in *Minimap2* (`-k17 -w17 --secondary=no`) getestet. Dieser führte jedoch zu einem noch niedrigeren BUSCO-Wert von 77,6 % für das *bacteria*-Set. Aus diesem Grund wurde im weiteren Verlauf auf die Standardparameter zurückgegriffen. Anschließend wurde das Racon-polierte Assembly mithilfe von *Medaka* weiter verfeinert, wobei ebenfalls die gefilterten ONT-Daten verwendet wurden. Die BUSCO-Analyse nach der Medaka-Polierung ergab 77,6 % für das *bacteria*-Set und 73,0 % für das *xanthomonadales_odb10*-Set. Damit konnte durch *Medaka* keine Verbesserung erzielt werden. Im Gegenteil: die Vollständigkeit nahm leicht ab. Zusammenfassend wurde in diesem Versuch dasselbe Flye-Assembly wie im vorherigen verwendet. Für die Polierung mit Racon und Medaka kamen jedoch ausschließlich die gefilterten ONT-Daten zum Einsatz. Die BUSCO-Werte fielen insgesamt etwas besser aus als bei der Verwendung der Rohdaten, jedoch zeigten weder *Racon* noch *Medaka* eine Verbesserung der Assemblierungsqualität, sondern eher eine Verschlechtung. Der Einsatz der strengeren Mapping-Parameter in *Minimap2* erwies sich ebenfalls als nachteilig und wurde daher nicht weiterverfolgt.

3.10 Flye-Assemblierung ohne interne Politur, anschließend verfeinert mit Racon, Medaka und Pilon

In einem weiteren Versuch wurde eine Assemblierung mit Flye durchgeführt, wobei als Eingangsdaten die auf 500 Millionen Basen gefilterten ONT-Rohdaten verwendet wurden. Dabei kamen die Standardparameter zum Einsatz, das heißt der Modus `meta`, eine minimale Überlappung von 5000 Basen sowie eine deaktivierte Polierung durch `iterations 0`, was dem Verhalten von *Flye* vor der letzten Softwareaktualisierung entspricht. Das resultierende Assembly zeigte zunächst einen BUSCO-Wert von 50,0 % für das *bacteria*-Set sowie 45,9 % für das *xanthomonadales_odb10*. Für die erste Polierung wurde *Racon* eingesetzt, wobei die gefilterten ONT-Daten zur Erstellung des Mappings genutzt wurden. Nach dieser Politur stiegen die BUSCO-Werte deutlich an und erreichten 75,9 % für *bacteria* sowie 72,7 % für *xanthomonadales_odb10*. Anschließend wurde eine weitere Verfeinerung mit *Medaka* vorgenommen. Hierbei kamen im Gegensatz zur vorherigen Polierung die ungefilterten ONT-Rohdaten zum Einsatz. Für das *bacteria*-Set ergab sich dabei eine leichte Verbesserung auf 76,7 %, während der Wert für *xanthomonadales_odb10* mit 72,7 % konstant blieb. Im letzten Schritt wurde eine Polierung mit *Pilon* durchgeführt. Dafür wurden die zuvor getrimmten Illumina-Daten verwendet. Diese führte zu einer signifikanten Qualitätssteigerung des Assemblies mit einem BUSCO-Wert von 96,6 % für das *bacteria* sowie 98,7 % für das *xanthomonadales_odb10*.

Zusammenfassend lässt sich feststellen, dass die Kombination aus einer *Flye*-Assemblierung ohne interne Politur sowie einer nachgelagerten Korrektur mit *Racon* und *Medaka* gefolgt von einer abschließenden Politur mit *Pilon* zu einer deutlichen Verbesserung der genomischen Vollständigkeit führte. Wurde hingegen die interne Politur in *Flye* aktiviert, konnten zwar ebenfalls hohe BUSCO-Werte erreicht werden, jedoch zeigte sich, dass zusätzliche Korrekturschritte mit *Racon* oder *Medaka* in diesem Fall tendenziell zu einer Verschlechterung der Vollständigkeit führten. Dies legt nahe, dass bei Verwendung der internen Politur von *Flye* auf weitere Feinkorrekturen vor der *Pilon*-Politur möglicherweise verzichtet werden kann.

Die folgenden Tabellen fassen die zentralen Ergebnisse der Assemblierungs- und Polierschritte aus den Schritten 3.6, 3.8, 3.9 und 3.10 zusammen. Dabei wurden alle Assemblierungen mit einer Abdeckung von 500 Mb durchgeführt, wobei unterschiedliche Polierungsiterationen bei Flye getestet wurden. Anschließend erfolgten die notwendigen Polierungsschritte mit den Tools Medaka oder Racon.

Tabelle 5: Vergleich der Auswirkungen unterschiedlicher Anzahl von Flye-Polishing-Iterationen

Schritt	**3.6**	**3.8**	**3.9**	**3.10**
Filtlong Filterung	durchgeführt	durchgeführt	durchgeführt	durchgeführt
Flye Assembly (Iteration)	`--iter 1`	`--iter 2`	`--iter 2`	`--iter 0`
BUSCO vor Racon (%)	88,8 %	88,8 %	88,8 %	50,0 %
Racon Polishing	nicht durchgeführt	1 Runde	1 Runde	1 Runde
BUSCO nach Racon (%)	—	69,8 %	78,5 %	75,9 %
Medaka Polishing	durchgeführt	durchgeführt	durchgeführt	durchgeführt
BUSCO nach Medaka (%)	88,8 %	81,0 %	77,6 %	76,7 %
Pilon Polishing	durchgeführt	durchgeführt	nicht durchgeführt	durchgeführt
BUSCO nach Pilon (%)	97,4 %	97,4 %	—	96,6 %
Medaka Runde 2	89,7 %	—	—	—
Medaka Runde 3	89,7 %	—	—	—
Pilon (nach Medaka Runde 2)	97,4 %	—	—	—

[1]Alle BUSCO-Werte beziehen sich auf die bacteria-odb12-Datenbank

Tabelle 6: Vergleich der Auswirkungen unterschiedlicher Anzahl von Flye-Polishing-Iterationen

Schritt	**3.6**	**3.8**	**3.9**	**3.10**
Filtlong	durchgeführt	durchgeführt	durchgeführt	durchgeführt
Flye Assembly	`--iter 1`	`--iter 2`	`--iter 2`	`--iter 0`
BUSCO vor Politur (%)	82,8 %	83,0 %	83,0 %	45,9 %
Racon	nicht durchgeführt	1 Runde	1 Runde	1 Runde
BUSCO nach Racon (%)	—	64,4 %	74,9 %	72,7 %
Medaka	durchgeführt	durchgeführt	durchgeführt	durchgeführt
BUSCO nach Medaka (%)	82,6 %	72,9 %	73,0 %	72,7 %
Pilon	durchgeführt	durchgeführt	nicht durchgeführt	durchgeführt
BUSCO nach Pilon (%)	98,8 %	98,2 %	—	98,7 %
Medaka Runde 2	82,6 %	—	—	—
Medaka Runde 3	82,6 %	—	—	—
Pilon (nach Medaka Runde 2)	98,9 %	—	—	—

[1]Alle BUSCO-Werte beziehen sich auf die Xanthomonadales-ODB10-Datenbank.

3.11 ONT-Assembly mit Polieren durch PacBio-Daten – technische Herausforderungen bei der Tool-Installation und Datenkonvertierung

Ein weiterer Ansatz bestand darin, ein Assembly basierend auf ONT-Daten zu erstellen und dieses anschließend mit PacBio-Daten zu polieren. Dafür wurde zunächst versucht, die für PacBio Werkzeuge zu installieren. Die Sammlung *pbbioconda*, welche diverse PacBio-spezifische Tools enthält, ist über den Bioconda-Kanal verfügbar und wurde entsprechend installiert. Für die Nutzung von *Arrow*, einem Polishing-Tool von PacBio, ist es erforderlich, dass die Sequenzdaten im BAM-Format vorliegen. Da die ursprünglich verfügbaren PacBio-Daten im alten *bax.h5*-Format vorlagen, wurde ein Konvertierungsschritt mittels *bax2bam* notwendig. Allerdings trat bei der Installation von *bax2bam* eine Fehlermeldung auf, und es zeigte sich, dass sowohl die Software selbst als auch die zugehörige Dokumentation veraltet oder nicht mehr verfügbar sind. Dies wurde unter anderem durch Hinweise aus einschlägigen Foren wie Biostars bestätigt. Auch alternative Werkzeuge wie *pbmm2* setzen das BAM-Format voraus, sodass das Problem der inkompatiblen Dateiformate weiterhin bestehen blieb. Es wurde darüber hinaus versucht, die offizielle Softwareplattform *SMRT Link* von PacBio herunterzuladen. Der Downloadprozess brach jedoch mehrfach ab, sodass auch dieser Weg nicht zum Ziel führte. Insgesamt konnte dieser Polierungsansatz mit PacBio-Daten aufgrund technischer Hürden bislang nicht erfolgreich umgesetzt werden. Die Nutzung von *SMRT Link* stellt grundsätzlich eine gute Option dar, scheiterte in diesem Fall jedoch an der fehlgeschlagenen Bereitstellung der Software.

3.12 ONT-Assemblierung mit Raven und anschließender Politur mit Racon, Medaka und Pilon

Ein weiterer Versuch basierte auf dem Einsatz von *Raven*, einem minimalistisch gehaltenen Assembler, der im Unterschied zu *Flye* über nur wenige einstellbare Parameter verfügt und keine eigene Polierung durchführt. Daher ist eine anschließende Polierung des Assemblies zwingend erforderlich. Es wurde *Raven* in der Version 1.8.3 installiert. Das Assembly erfolgte mit den gefilterten 500M Abdeckng ONT-Daten und konnte sehr schnell abgeschlossen werden. Die erste *BUSCO*-Analyse des *Raven*-Assemblies ergab eine Vollständigkeit von 72,4 % für das *bacteria*, Für das *xanthomonadales_odb10* lag der BUSCO-Wert bei 66,2 %. Zur Politur wurde anschließend *Racon* eingesetzt. Das Mapping erfolgte mit *minimap2* unter Nutzung der gefilterten ONT-Daten. Nach der ersten Politur mit *Racon* konnte eine deutliche Verbesserung erzielt werden: Der BUSCO-Wert stieg auf 77,6 % für das *bacteria* und 74,0 % für das *xanthomonadales_odb10*. Im Anschluss wurde eine weitere Verfeinerung mit *Medaka* durchgeführt, wobei diesmal die Rohdaten zum Einsatz kamen. Hierdurch verbesserte sich der BUSCO-Wert geringfügig auf 78,4 % für das *bacteria*-Set und auf 74,6 % für das *xanthomonadales_odb10*. Zur finalen Qualitätssteigerung wurde das medaka-polierte Assembly mit *Pilon* unter Verwendung der getrimmten Illumina-Daten weiter poliert. Bereits nach einer Runde mit *Pilon* erreichte der BUSCO-Wert 97,4 % für das *bacteria*-Set und 98,7 % für das *xanthomonadales_odb10*-Set. Eine zweite Runde mit *Pilon* führte schließlich zu einer leichten weiteren Verbesserung, insbesondere beim *xanthomonadales*-Set, dessen BUSCO-Wert auf 98,9 % anstieg, während der Wert für das *bacteria*-Set konstant bei 97,4 % blieb. Diese Ergebnisse zeigen, dass trotz der minimalen Grundfunktionalität von *Raven* durch gezielte Nachbearbeitung mit *Racon*, *Medaka* und *Pilon* eine sehr hohe Assembly-Qualität im Hinblick auf den BUSCO-Score erreicht werden kann.

3.13 ONT-Assemblierung mit Canu und Politur mit Racon, Medaka und Pilon

Ein weiterer Ansatz zur Assemblierung der ONT-Daten wurde mit dem Assembler *Canu* (Version 2.2) getestet. Nach anfänglichen Problemen beim Starten des Jobs ("`client_loop: send disconnect: Broken pipe`") wurde der Befehl im Hintergrund mit nohup erneut gestartet. Das Assembly erfolgte mit den gefilterten ONT Daten. Dabei wurde das Parameter genomeSize =5m genutzt. Die erste *BUSCO*-Analyse des *Canu*-Assemblies ergab für das *xanthomonadales_odb10* eine Vollständigkeit von 69,4 % Für das allgemeine *bacteria*lag die Vollständigkeit bei 75 %. Zur Verbesserung der Assemblierungsqualität wurde das *Canu*-Assembly mit *Racon* poliert. Dabei wurden die ONT-Rohdaten mit *minimap2* gegen das Assembly gemappt und anschließend mit *Racon* verarbeitet. Die anschließende BUSCO-Analyse zeigte jedoch einen leichten Rückgang der Werte: Für das *xanthomonadales*-Set sank der BUSCO-Wert auf 65,5 %, für das *bacteria*-Set auf 69 %. Im nächsten Schritt wurde das racon-polierte Assembly mit *Medaka* weiterverarbeitet. Dies führte zu einer Verbesserung: Die BUSCO-Vollständigkeit stieg auf 73,4 % (xanthomonadales) bzw. 78,4 % (bacteria). Zur finalen Politur kam erneut *Pilon* unter Nutzung getrimmter Illumina-Reads zum Einsatz. Nach der ersten Runde stieg die BUSCO-Vollständigkeit deutlich an: 96,6 % für *bacteria* und 98,4 % für *xanthomonadales*. Zwei weitere Runden mit *Pilon* führten zu einer leichten weiteren Verbesserung: In der zweiten Runde lag die BUSCO-Vollständigkeit bei 97,4 % (*bacteria*) und 98,9 % (*xanthomonadales*), die dritte Runde brachte keine weitere Veränderung.

Tabelle 7: Vergleich der Assemblierung mit Raven (3.12) und Canu (3.13) sowie Einfluss von Roh- versus gefilterten Daten auf das Polishing1

Verarbeitungsschritt	**3.12**	**3.13**
Assemblierung durchgeführt	ja	ja
BUSCO Bakterien (%)	72,4 %	75,0 %
BUSCO Xanthomonadales (%)	66,2 %	69,4 %
Racon Politur	ja, gefilterte ONT-Daten	ja, Rohdaten ONT
BUSCO Bakterien (%)	77,6 %	69,0 %
BUSCO Xanthomonadales (%)	74,0 %	65,5 %
Medaka Politur	ja, Rohdaten ONT	ja, Rohdaten ONT
BUSCO Bakterien (%)	78,4 %	78,4 %
BUSCO Xanthomonadales (%)	74,6 %	73,4 %
Pilon Politur Runde 1	ja, gefilterte Illumina-Daten	ja, gefilterte Illumina-Daten
BUSCO Bakterien (%)	97,4 %	96,6 %
BUSCO Xanthomonadales (%)	98,7 %	98,4 %
Pilon Politur Runde 2	ja	ja
BUSCO Bakterien (%)	97,4 %	97,4 %
BUSCO Xanthomonadales (%)	98,9 %	98,9 %
Pilon Politur Runde 3	nein	ja
BUSCO Bakterien (%)	—	97,4 %
BUSCO Xanthomonadales (%)	—	98,9 %

Zur Vollständigkeit und für einen direkten Vergleich der Auswirkungen von Rohdaten gegenüber gefilterten ONT-Daten wurde zusätzlich eine *Raven*-Assemblierung mit *Racon* poliert, diesmal unter Verwendung der nicht gefilterten Rohdaten. Dabei zeigte sich eine Verringerung der BUSCO-Vollständigkeit im Vergleich zur Politur mit gefilterten ONT-Reads. Der gleiche Effekt konnte auch bei der *Canu*-Assemblierung beobachtet werden, bei der ebenfalls niedrigere BUSCO-Werte bei Verwendung der Rohdaten im Vergleich zu gefilterten Daten erzielt wurden. Bei der Anwendung von *Medaka* wurden in diesem Fall ausschließlich gefilterte ONT-Daten anstelle der Rohdaten verwendet. Diese Ergebnisse sind in Tabelle 8 dargestellt.

Tabelle 8: Vergleich der Assemblierung mit Raven (3.12) und Canu (3.13) sowie Einfluss von Roh- versus gefilterten Daten auf das Polishing2

Verarbeitungsschritt	**3.12**	**3.13**
Assemblierung durchgeführt	ja	ja
BUSCO Bakterien (%)	72,4 %	75,0 %
BUSCO Xanthomonadales (%)	66,2 %	69,4 %
Racon-Politur	ja, Rohdaten ONT	ja, gefilterte ONT
BUSCO Bakterien (%)	67,2 %	79,3 %
BUSCO Xanthomonadales (%)	64,8 %	74,4 %
Medaka-Politur	ja, gefilterte ONT	ja, gefilterte ONT
BUSCO Bakterien (%)	75,0 %	76,7 %
BUSCO Xanthomonadales (%)	71,4 %	72,9 %

3.14 Hybrid-Assemblierung mit Unicycler unter Nutzung von Illumina- und ONT-Daten

Es wurde eine Hybrid-Assemblierung mit der Software *Unicycler* (Version 0.5.1) durchgeführt. Ziel war es, Illumina-Short-Reads mit ONT-Long-Reads zu kombinieren, um eine möglichst vollständige und genaue Genomassemblierung zu erreichen. *Unicycler* bietet dafür drei Modi – *conservative*, *normal* und *bold*. Laut Literatur wird der Modus *normal* empfohlen, da er eine gute Balance zwischen konservativer Assemblierung und aggressiver Kontig-Verlängerung bietet.
Im ersten Lauf, bei dem die Rohdaten der ONT-Sequenzierung verwendet wurden, kam es zu einem Abbruch mit der Fehlermeldung „Getötet“ während der Overlap-Berechnung mit *minimap*, was vermutlich auf unzureichenden RAM zurückzuführen ist. Daher wurde ein zweiter Versuch mit den gefilterten ONT-Daten durchgeführt, kombiniert mit den Rohdaten der Illumina-Paired-End-Sequenzierung. Die Assemblierung verlief in diesem zweiten Anlauf erfolgreich. Die anschließende Qualitätsbewertung mit *BUSCO* ergab eine Vollständigkeit von **97,4 %** für das allgemeine *bacteria*-Referenzset und **98,9 %** für das *xanthomonadales_odb10*-Set. Trotzdem wurde ein weiterer Versuch unternommen, das Ergebnis mithilfe von *Medaka* weiter zu verbessern. Dabei wurden erneut die ungefilterten ONT-Daten zur Politur des Assemblats verwendet. Nach der Konsensbildung mit *Medaka* zeigte die *BUSCO*-Analyse jedoch eine geringere Vollständigkeit: **94,8 %** für das *bacteria*-Set sowie **96,4 %** für das *xanthomonadales_odb10*-Set. Die zusätzliche Politur mit *Medaka* führte somit zu einer Verschlechterung der Genomvollständigkeit. Daher wird die ursprüngliche Assemblierung durch *Unicycler* ohne weitere Nachbearbeitung als die qualitativ beste Version angesehen.

Tabelle 9: Ergebnisse der Unicycler-Hybrid-Assemblierung

Schritt	**Datengrundlage**	**BUSCO (Bakterien) [%]**	**BUSCO (Xanthomonadales) [%]**
Unicycler	Illumina Rohdaten + ONT Rohdaten	—	—
Unicycler	Illumina Rohdaten + ONT gefiltert	97,4 %	98,9 %
Medaka Polish	ONT Rohdaten	94,8 %	96,4 %

3.15 Politur mit Homopolish auf Basis der Flye-Assemblierung aus Abschnitt 3.6 und abschließender Pilon-Polierung

In diesem Versuch wurde das Tool *Homopolish* (Version 0.4.1) eingesetzt, um die bestehende *Flye*-Assemblierung aus Abschnitt 3.6, basierend auf ONT-Daten, weiter zu optimieren. Ziel war es, insbesondere homopolymerbedingte Fehler zu korrigieren, wie sie typischerweise bei Oxford-Nanopore-Sequenzierungen auftreten. Die erste Polierung erfolgte mithilfe einer Referenzdatenbank für *Xanthomonas oryzae*. Ein zusätzlicher Versuch, eine generische bakterielle MASH-Datenbank (für Bakterien allgemein) zur Referenzierung zu verwenden, war hingegen nicht erfolgreich, da die benötigten Sketche unter dem entsprechenden Repository-Link nicht mehr verfügbar sind (`https://github.com/ythuang0522/homopolish/tree/v0.4.1?tab=readme-ov-file`, Abschnitt "Download virus, bacteria, or fungi sketches"). Daraufhin wurde ein weiterer Durchlauf mit der allgemeinen Gattungsreferenz *Xanthomonas* durchgeführt. Die Qualität der resultierenden Assemblierungen wurde anschließend mit BUSCO bewertet. Die Assemblierung, die auf der allgemeinen Gattung *Xanthomonas* basierte, erreichte einen BUSCO-Vollständigkeitswert von 89,7 % bei Nutzung des bakteriellen Referenzsets und 82,8 % bei Nutzung der *xanthomonadales_odb10*-Referenz. Deutlich bessere Ergebnisse konnten mit der spezifischeren Referenz *Xanthomonas oryzae* erzielt werden. Hier lag der BUSCO-Wert bei 97,4 % für das bakterielle Referenzset und bei 97,6 % für die *xanthomonadales_odb10*-Referenz.

Aufbauend auf dem besten Ergebnis wurde das durch Homopolish erzeugte Assemblierung in einem weiteren Schritt mithilfe von Illumina-Daten und *Pilon* weiter poliert. Die finale Bewertung der so entstandenen Assemblierung ergab einen BUSCO-Wert von 98,9 % gegenüber der *xanthomonadales_odb10*-Referenz, was eine Verbesserung darstellt.

Tabelle 10: BUSCO-Ergebnisse für *xan* und *oryzae* vor und nach Polishing

	Rohassembly		**Nach Polishing**		
Stamm	**Bakterien [%]**	**Xanthomonadales [%]**	**Werkzeug**	**Bakterien [%]**	**Xanthomonadales [%]**
xan	89,7	82,8	`Pilon`	–	–
oryzae	97,4	97,6	`Pilon`	97,4	**98,9**

Datenquelle: `https://ftp.ncbi.nlm.nih.gov/genomes/refseq/bacteria/Xanthomonas_oryzae/`

3.16 Hybridassemblierung mit MaSuRCA

Für die Hybridassemblierung wurde *MaSuRCA* (Version 4.1.3) mit den Standardeinstellungen verwendet. Die Assemblierung erfolgte durch die Erstellung einer Konfigurationsdatei und anschließendes Ausführen des Assemblierungsskripts. Die Qualität der Assemblierung wurde mit BUSCO bewertet. Dabei zeigte sich für das Xanthomonadales-Datenbank-Set eine Vollständigkeit von 98,9 %, während bei der allgemeinen Bakterien-Datenbank 97,4 % der erwarteten Gene gefunden wurden.

3.17 Illumina-basierte Assemblierung mit MaSuRCA

Für die Assemblierung mit Illumina-Daten wurde MaSuRCA mit den Standardeinstellungen verwendet. Die Assemblierung erfolgte durch Ausführen von masurca config.txt und anschließendem Start des Skripts assemble.sh. Während des Prozesses trat ein Fehler beim Gap Closing auf, dennoch konnte das Assembly aus den Pre-Gap-Close-Dateien im Verzeichnis CA/9-terminator/ verwendet werden. Die fertige Assembly-Datei liegt unter CA/9-terminator/genome.scf.fasta vor. Die Qualität der Assemblierung wurde mit BUSCO bewertet. Dabei zeigte sich bei Verwendung des Xanthomonadales-Datenbanksatzes eine Vollständigkeit von 98,9 %. Es zeigt sich, dass eine gute Assemblierung hinsichtlich BUSCO nur mit Illumina-Daten möglich ist. Daher wurde eine strenge Filterung der ONT-Daten auf 150 Millionen Basen mit filtlong durchgeführt, um diese für eine Polierung zu nutzen. Die Polierung erfolgte mit *Medaka* unter Verwendung der gefilterten ONT-Daten und dem Illumina-basierten Assembly. Nach der Polierung sank die Vollständigkeit bei BUSCO auf 84,3 % im Xanthomonadales-Datensatz.

3.18 Polierung der mit Flye assemblierten ONT-Daten mittels Computational Correction

In diesem Versuch wurde die bestehende *Flye*-Assemblierung aus Abschnitt 3.6, basierend auf ONT-Daten, mithilfe des Tools *Computational Correction* weiter korrigiert und poliert. Für die Korrektur der TALEs wurde zunächst das erste Skript vom Tool verwendet. Im nächsten Schritt wurde das Mapping sowohl mit den Rohdaten als auch mit den gefilterten ONT-Daten durchgeführt. Für beide Varianten wurde anschließend das zweite Bash-Skript zur Polierung eingesetzt. Zur Qualitätsbewertung wurden BUSCO-Analysen durchgeführt. Für die mit den Rohdaten polierte Version ergaben sich 98,9 % vollständige Gene im Xanthomonadales-Datensatz und 97,4 % im allgemeinen Bakterien-Datensatz. Identische Werte wurden auch für die Variante mit den gefilterten ONT-Daten beobachtet. Zusätzlich wurde geprüft, wie sich die BUSCO-Werte verhalten, wenn ausschließlich der erste Schritt, also die reine Korrektur ohne Polierung durchgeführt wird. Das Ergebnis zeigte ebenfalls eine Vollständigkeit von 98,9 % im Xanthomonadales-Datensatz. Darüber hinaus blieben auch weitere Qualitätsmetriken wie der N50-Wert sowie die Anzahl der später mit AnnoTALE annotierten Gene und Pseudogene unverändert. Dies deutet darauf hin, dass die Polierungsschritte in diesem Fall keine Verbesserung der Assemblierungsqualität erbracht haben.

3.19 Annotation mit AnnoTALE

Für die Identifikation und Klassifikation von TALE-Genen wurde das Tool *AnnoTALE* verwendet. Die Anwendung erfolgte in mehreren Schritten:

Zunächst wurden TALE-Gene auf Basis der vorhandenen Assemblierung vorhergesagt. Anschließend wurden die vorhergesagten Sequenzen analysiert, um Merkmale und Wiederholungen innerhalb der TALE-Domänen zu charakterisieren. In einem weiteren Schritt erfolgte die Zuordnung der TALEs zu bestehenden Klassen mithilfe einer Klassifizierungsdatei, die auf bekannten Referenzsequenzen basiert. Die Analysen wurden sowohl mit als auch ohne Aktivierung des *Sensitive Mode* durchgeführt, um mögliche Unterschiede in der Erkennung sensitiv exprimierter oder stark divergenter TALE-Gene zu erfassen. Es zeigte sich jedoch, dass beide Modi identische Ergebnisse lieferten, sodass der *Sensitive Mode* in diesem Fall keinen Einfluss auf die Annotationsergebnisse hatte.

Bei der Durchführung der Analyse mit *AnnoTALE* traten bei drei Assemblierungen Fehlermeldungen auf, die zu einem Abbruch des Programms führten. Konkret handelte es sich dabei um eine `NullPointerException` sowie eine `NegativeArraySizeException (-11)`. Zwei dieser fehlerhaften Assemblierungen stammen aus den Abschnitten 3.14 und 3.17, bei denen sehr kurze Contigs vorlagen, was vermutlich die Ursache für die auftretenden Fehler darstellte. Die dritte betroffene Assemblierung stammt aus Abschnitt 3.3, hier konnten keinerlei TALE-Gene annotiert werden, was ebenfalls zum Abbruch führte. Zur Untersuchung der aufgetretenen Probleme wurde das Tool *QUAST* (Version 5.3.0) eingesetzt. Dabei wurden sehr kurze Reads aus den FASTA-Dateien entfernt. Anschließend wurde die Analyse mit *AnnoTALE* erneut durchgeführt, wodurch die zuvor beobachteten Fehlermeldungen behoben werden konnten.

Bei einigen Pseudogenen konnte keine Proteinübersetzung erzeugt und keine Analyse durch Aufteilung in Regionen durchgeführt werden. Ursache hierfür sind Lücken (Gaps) an RVD-Positionen, die die korrekte Proteinkodierung verhindern. Für diese Fälle sind die entsprechenden DNA-Sequenzen in der Datei TALE_DNA_parts.fasta enthalten, während keine Proteinsequenzen in TALE_Protein_parts.fasta vorliegen.

In der nachfolgenden Tabelle 11 sind die BUSCO-Scores basierend auf der Xanthomonadales-Datenbank für jede Assemblierung aufgeführt. Zusätzlich werden die verwendeten Sequenzierungsdaten, das eingesetzte Assemblierungstool sowie die jeweilige Polishing-Strategie dargestellt. Die ergänzende Tabelle 12 fasst die N50-Werte sowie die Ergebnisse der Genomannotation zusammen. Hier werden insbesondere die Anzahl der identifizierten TALEs und Pseudogene für jede Assemblierung aufgeführt.

Tabelle 11: Übersicht der Assemblierungs- und Polierschritte mit BUSCO-Ergebnissen (Xanthomonadales)

Abschnitt	**Technologie**	**Assembler**	**Polishing**	**BUSCO Score**
3.4	Illumina	SPAdes	Medaka (ONT)	98.9
3.6	ONT (500 Mb)	Flye (iter 1)	Illumina (pilon)	98.8
3.7	ONT (350 Mb)	Flye (iter 1)	Illumina (pilon)	98.9
3.8	ONT (500 Mb)	Flye (iter 2)	Illumina (pilon)	98.2
3.10	ONT (500 Mb)	Flye (iter 0)	Illumina (pilon)	98.7
3.12	ONT	Raven	Illumina (pilon)	98.9
3.13	ONT	Canu	Illumina (pilon)	98.9
3.14	ONT + Illumina	Unicycler	Medaka	98.9
3.15	ONT	Flye	Homopolish (Xanthomonas)	82.8
3.15	ONT	Flye	Homopolish (X. oryzae)	97.6
3.15	ONT	Flye	Homopolish (X. oryzae) + pilon	98.9
3.16	ONT + Illumina	MaSuRCA	-	98.9
3.17	Illumina	MaSuRCA	-	98.9
3.18	ONT	Flye	comp. correction	98.9

Tabelle 12: Vergleich von N50-Werten, TALE-Annotationen und Pseudogenen

Abschnitt	**N50-Wert**	**TALEs**	**Pseudogene**
3.4	24,403	0	0
3.6	4,982,411	5	16
3.7	4,989,843	15	6
3.8	4,989,985	12	9
3.10	4,989,842	13	8
3.12	4,989,794	17	4
3.13	5,050,764	13	8
3.14	4,964,981	8	9
3.15	4,986,754	0	21
3.15	4,989,692	15	6
3.15	4,989,892	18	3
3.16	4,997,576	1	18
3.17	25,362	1	2
3.18	4,989,950	20	1

3.20 Visualisierung mit ProgressiveMauve

Vor der Visualisierung wurde der jeweils längste Contig in den beiden mit Illumina-polierten ONT-Assemblierungen (500 Mb und 350 Mb) mithilfe von Biopython in seine Reverse-Komplement-Sequenz überführt und ersetzt. Dieser Schritt ermöglicht eine sinnvolle Orientierung der Sequenz und erleichtert die nachfolgende Analyse, den Vergleich sowie die grafische Darstellung der Assemblierungsergebnisse.

Die drei in der folgenden Abbildung dargestellten Assemblierungen umfassen:

1. `flye_pilon_500` aus der referenzierten Publikation (verfügbar unter `https://zenodo.org/records/6854866`),
2. `flye_pilon_500` aus der vorliegenden Studie, 3.6
3. `flye_pilon_350`, ebenfalls aus der vorliegenden Studie, 3.7

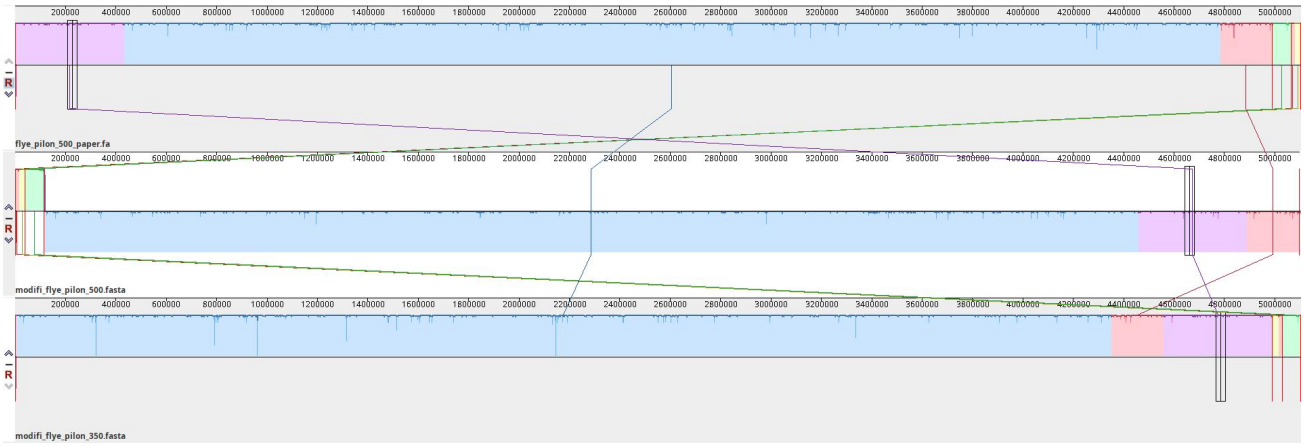


Abbildung 1: Vergleich dreier Flye/Pilon-Assemblierungen auf Basis von ONT-Daten bei unterschiedlicher Abdeckung, visualisiert mit progressiveMauve

Im folgenden Bild sind die Varianten mit der comp. correction dargestellt:

1. die ONT-Assemblierung mit der comp. correction aus der Publikation (verfügbar unter `https://zenodo.org/records/6854866`),
2. Korrektur ohne Polierung aus der vorliegenden Studie, 3.18
3. Korrektur mit Polierung und Nutzng der gefilterten ONT-Daten aus der vorliegenden Studie, 3.18
4. Korrektur mit Polierung und Nutzng der ONT-Rohdaten aus der vorliegenden Studie, 3.18

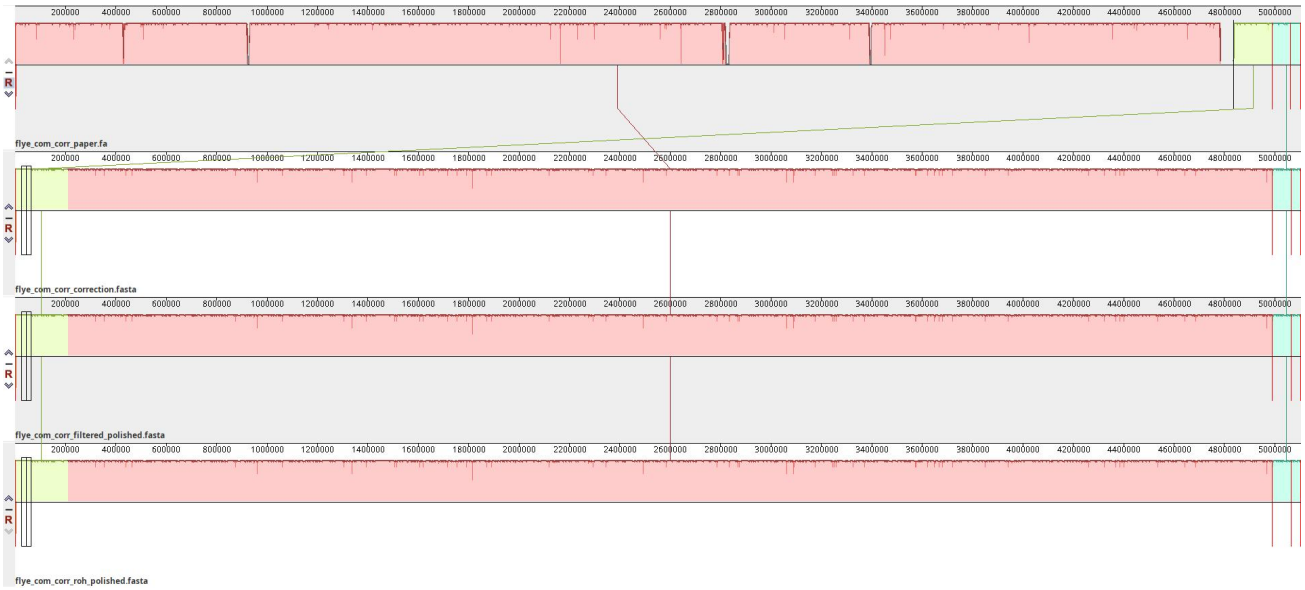


Abbildung 2: Vergleich von ONT-Assemblierungen mit unterschiedlichen Korrekturmethoden: (1) aus Publikation, (2) ohne Polierung, (3) mit Polierung und Nutzung gefilterter ONT-Daten, (4) mit Polierung und Nutzung von ONT-Rohdaten.

Eine zusammenfassende Darstellung aller in den vorherigen zwei Abbildungen gezeigten Fälle findet sich in Abbildung 3.



Abbildung 3: Übersichtliche Zusammenfassung aller in den vorherigen Abbildungen dargestellten Assemblierungen

3.21 Pseudogene Alignments

Zur Untersuchung möglicher Ursachen für das Auftreten von Pseudogenen in den Assemblierungen sowie zur Analyse der Unterschiede zwischen Genen und Pseudogenen wurden drei TALE-Gene exemplarisch betrachtet. Hierzu wurden Sequenzen aus zwei Assemblierungen miteinander verglichen: einerseits aus Abschnitt 3.6 (ONT-Assemblierung mit anschließender Politur durch Pilon), in der die betreffenden Sequenzen als Pseudogene annotiert wurden, und andererseits aus Abschnitt 3.18 (Assemblierung mit Computergestützte Korrektur), in der dieselben Sequenzen als funktionale Gene vorlagen. Durch gezieltes Alignment sollten positionsspezifische Unterschiede identifiziert und potenzielle Ursachen für fehlerhafte Genannotationen aufgezeigt werden.
Die resultierenden Alignments der drei untersuchten TALE-Beispiele sind im Anhang als Abbildungen dargestellt (siehe Anhang A). Zur besseren Übersicht wurden diese auf Proteinebene ausgerichtet. Zusätzlich wurde ein Beispiel (TALE5) exemplarisch auf DNA-Ebene mit T-Coffee aligniert; dieses Alignment ist im Anhang B zu finden. Dabei ist in den Alignments zu erkennen, dass die Pseudogene durch Lücken (Gaps) sowohl am Anfang als auch am Ende der Sequenzen gekennzeichnet sind, also im Bereich des N- und C-Terminus. Diese fehlenden Sequenzabschnitte könnten die Ursache für die Annotation als Pseudogene sein.

3.22 Analyse des einzigen TALE-Pseudogens nach comp. correction

Im Abschnitt 3.18 wurde im Rahmen der Analyse mit comp. correction ein Pseudogen identifiziert. Um dieses näher zu untersuchen, wurde die entsprechende Proteinsequenz mit anderen TALE-Genen aligniert. Dabei zeigte sich, dass das Pseudogen im Vergleich zu allen anderen TALE-Genen ein deutlich verkürztes C-Terminus aufweist, was charakteristisch für iTALEs ist. Diese stellen eine funktionell abweichende Untergruppe der TALEs dar, da sie nicht zur Genaktivierung beitragen, sondern stattdessen die Erkennung durch bestimmte pflanzliche Resistenzproteine unterbinden.

```
>tempTALE4 [Pseudo] [2458990-2462005:1]: C-terminus
SIVAQLSRRDPALAALTNDHLVALACLGGRPALDAVKKGLPHAPELIRRVNSRIGERTSH
RVADLAHVVRVLGFFQSHSHPAQAFDDAMTQFGMSRHGLVQLFRRVGVTEFEARCGTIPP
ASQRWDRILQASGTKRAKPSPTSAQTPDQASLHAFPDSLERDLDAPSPMHEGDQTRASRR
KRS*----------------------------------------------------------
-------------------------------------------
```

4 Disskusion

Wie in Abschnitt 3.2 gezeigt wurde, ist eine gründliche Qualitätskontrolle sowie eine gezielte Filterung der Rohdaten essenziell, um eine verlässliche Grundlage für den Assemblierungsprozess zu schaffen. Eine gute Ausgangsqualität der Daten hat sich dabei als entscheidend für die spätere Assemblierungsqualität erwiesen. Auch bei der Polierung mit Medaka oder Racon zeigt sich, dass die Auswahl zwischen ungefilterten und gefilterten Rohdaten einen Einfluss auf die Vollständigkeit der Assemblierung haben kann, wie in den Abschnitten 3.12 und 3.13 dargestellt. In diesen Fällen führten die gefilterten Daten zu höheren BUSCO-Werten. Diese Ergebnisse sollten jedoch mit Vorsicht interpretiert werden, da eine zu starke Filterung potenziell zur Eliminierung biologisch relevanter Sequenzen führen und somit die Gesamtqualität der Assemblierung beeinträchtigen kann. Diese möglichen Qualitätseinbußen werden durch die BUSCO-Werte allein nicht vollständig erfasst. Um eine umfassendere Bewertung der Assemblierungsqualität zu ermöglichen, ist daher auch eine funktionelle Annotation notwendig, ebenso wie die Berücksichtigung weiterer Assemblierungsmetriken. Ein weiterer zentraler Aspekt ist, dass in früheren Arbeiten, in denen ältere Versionen von Flye verwendet wurden, noch keine integrierte Polierung enthalten war. Daher war es erforderlich, die Assemblierungen anschließend mit Tools wie Medaka oder Racon zu polieren, um die Genauigkeit zu verbessern. In der aktuellen Flye-Version ist jedoch bereits eine interne Polierung integriert. Eine zusätzliche Anwendung von Medaka oder Racon kann unter diesen Umständen zu einer Verschlechterung der BUSCO-Werte führen. Dieser Effekt ist in den Tabellen 5 und 6 nachvollziehbar dargestellt. Weiterhin konnte beobachtet werden, dass durch die anschließende Polierung mit Pilon dieser Effekt weitgehend ausgeglichen wird. Unabhängig davon, welche BUSCO-Werte nach der ersten Polierung mit Medaka oder Racon erzielt wurden, konnte durch eine oder mehrere Runden Pilon-Polierung in allen Fällen ein vergleichbares hohes Niveau erreicht werden. Insgesamt zeigen alle durchgeführten Assemblierungen sehr gute BUSCO-Werte, unabhängig davon, wie vollständig die Assemblierung letztlich annotiert wurde oder ob sie eine hohe Anzahl an Pseudogenen enthält. Besonders auffällig ist, dass in den mit Illumina-polierten Assemblierungen eine erhöhte Anzahl an Pseudogenen identifiziert wurde. Im Gegensatz dazu wurde bei der Assemblierung mit Flye unter Anwendung der computational Correction, die speziell für die Korrektur von TALE-Genen entwickelt wurde, nahezu ausschließlich vollständige TALE-Gene annotiert. Lediglich ein einzelnes Pseudogen wurde detektiert, das sich nach genauer Analyse als iTALE klassifizieren ließ. Dieses Ergebnis bestätigt die Beobachtungen aus der referenzierten Publikation und zeigt, dass gezielte Polierung vor der Annotation besonders für TALE-Gene hilfreich ist.

4.1 Persönlicher Eindruck

Durch das Projekt konnten wertvolle Einblicke in die verschiedenen Schritte der bioinformatischen Analyse von Sequenzierdaten gewonnen werden. Insbesondere im Bereich der Qualitätskontrolle zeigten sich deutliche Auswirkungen unterschiedlicher Filterungsparameter auf nachfolgende Analyseprozesse. Auch das Zusammenspiel verschiedener Werkzeuge innerhalb der Assemblierungspipeline sowie die dabei auftretenden Abhängigkeiten von Tools und deren Versionen wurden durch die praktische Umsetzung deutlich. So war es möglich, technische Herausforderungen zu identifizieren und passende Lösungen gezielt umzusetzen. Die Arbeit mit verschiedenen Assemblierungs- und Polierungstools war teilweise zeitaufwendig, insbesondere wenn einzelne Schritte mehrere Versuche oder umfangreiche Rechenzeiten erforderten. Dennoch trug gerade diese Auseinandersetzung wesentlich zum Verständnis der methodischen Zusammenhänge bei. Durch die Annotation der TALE-Gene bot das Projekt wertvolle Einblicke in funktionelle Genannotationen und die Analyse spezieller Genfamilien. Fachlich und methodisch war es sehr bereichernd. Insgesamt war das Projekt fachlich sehr gewinnbringend und hat mir sowohl methodisch als auch technisch eine solide Grundlage für zukünftige Arbeiten in der Genom- und Bioinformatikforschung gegeben.

5 Literatur

Jan Grau, René Grove, Annett Erkes, Neele Unger, Marcel Göricke, and Jens Boch. Time-resolved rna-seq data reveal dynamic expressional behaviour of tale target genes. *bioRxiv*, 2024. doi: 10.1101/2024.05.28.596194. URL https://www.biorxiv.org/content/early/2024/05/28/2024.05.28.596194.

Stefanie Mücke, Maik Reschke, Annett Erkes, Claudia-Alice Schwietzer, Sebastian Becker, Jana Streubel, Richard D. Morgan, Geoffrey G. Wilson, Jan Grau, and Jens Boch. Transcriptional reprogramming of rice cells by xanthomonas oryzae tales. *Frontiers in Plant Science*, Volume 10 - 2019, 2019. ISSN 1664-462X. doi: 10.3389/fpls.2019.00162. URL https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2019.00162.

Annett Erkes, Maik Reschke, Jens Boch, and Jan Grau. Evolution of transcription activator-like effectors in xanthomonas oryzae. *Genome Biology and Evolution*, 9(6):1599–1615, 06 2017. ISSN 1759-6653. doi: 10.1093/gbe/evx108. URL https://doi.org/10.1093/gbe/evx108.

Annett Erkes, René P. Grove, Milena Žarković, Sebastian Krautwurst, Ralf Koebnik, Richard D. Morgan, Geoffrey G. Wilson, Martin Hölzer, Manja Marz, Jens Boch, and Jan Grau. Assembling highly repetitive xanthomonas talomes using oxford nanopore sequencing. *BMC Genomics*, 24(1):151, Mar 2023. ISSN 1471-2164. doi: 10.1186/s12864-023-09228-1. URL https://doi.org/10.1186/s12864-023-09228-1.

Taishan Hu, Nilesh Chitnis, Dimitri Monos, and Anh Dinh. Next-generation sequencing technologies: An overview. *unknown*, 2025.

Rainer Merkl. *Bioinformatik*. Wiley-VCH GmbH, 2022. ISBN 978-3-527-83390-0. E-Book (PDF), 751 Seiten.

Ivan Sovic. Racon: Ultrafast consensus module for raw de novo genome assembly of long uncorrected reads. https://github.com/isovic/racon, 2017. Abgerufen am 21. Juli 2025.

Oxford Nanopore Technologies. Medaka: Sequence correction provided by ont research. https://github.com/nanoporetech/medaka, 2019. Open-source software, abgerufen am 21.07.2025.

Bruce Walker and Brad Langhorst. Pilon: Automated genome assembly improvement and variant detection tool, 2015. URL https://github.com/broadinstitute/pilon. Abgerufen am 21. Juli 2025.

Jan Grau, Maik Reschke, Annett Erkes, Jana Streubel, Richard D. Morgan, Geoffrey G. Wilson, Ralf Koebnik, and Jens Boch. Annotale: bioinformatics tools for identification, annotation and nomenclature of tales from xanthomonas genomic sequences. *Scientific Reports*, 6:21077, Feb 2016. ISSN 2045-2322. doi: 10.1038/srep21077. URL https://doi.org/10.1038/srep21077.

Annekatrin Richter, Jana Streubel, Christina Blücher, Boris Szurek, Maik Reschke, Jan Grau, and Jens Boch. A tal effector repeat architecture for frameshift binding. *Nature Communications*, 5:3447, Mar 2014. ISSN 2041-1723. doi: 10.1038/ncomms4447. URL https://doi.org/10.1038/ncomms4447.

Michael Alonge. Ragtag: Tools for fast and flexible genome assembly scaffolding and improvement. https://github.com/malonge/RagTag, 2025. Accessed: 2025-08-03.

Heng Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018. doi: 10.1093/bioinformatics/bty191. URL https://academic.oup.com/bioinformatics/article/34/18/3094/4994778.

Pascal Hirsch and Alexey Gurevich. Webquast: Online evaluation of genome assemblies. *Nucleic Acids Research*, 51(W1):W601–W606, 2023. doi: 10.1093/nar/gkad406. URL https://academic.oup.com/nar/article/51/W1/W601/7167303.

Aaron E. Darling, Bob Mau, and Nicole T. Perna. Progressive mauve: Multiple alignment of genomes with gene flux and rearrangement. *PLOS ONE*, 5(6):e11147, 2010. doi: 10.1371/journal.pone.0011147. URL https://doi.org/10.1371/journal.pone.0011147.

Robert Vaser and Mile Šikić. Raven: A de novo genome assembler for long reads. *Nature Computational Science*, 1(6):333–340, 2021. doi: 10.1038/s43588-021-00073-4. URL https://doi.org/10.1038/s43588-021-00073-4.

Yao-Ting Huang, Po-Yu Liu, and Pei-Wen Shih. Homopolish: A method for the removal of systematic errors in nanopore sequencing by homologous polishing. *Genome Biology*, 22(1):95, 2021. doi: 10.1186/s13059-021-02282-6. URL https://genomebiology.biomedcentral.com/articles/10.1186/s13059-021-02282-6.

Ryan R. Wick. Filtlong: quality filtering tool for long reads. https://github.com/rrwick/Filtlong, 2017. Accessed: 2025-08-03.

Mikhail Kolmogorov, Jeffrey Yuan, Yu Lin, and Pavel A Pevzner. Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, 37(5):540–546, 2019. doi: 10.1038/s41587-019-0072-8.

Mosè Manni, Matthew R Berkeley, Mathieu Seppey, Felipe A Simão, and Evgeny M Zdobnov. Busco update: Novel and streamlined workflows along with broader and deeper phylogenetic coverage for scoring of eukaryotic, prokaryotic, and viral genomes. *Molecular Biology and Evolution*, 38(10):4647–4654, 2021. doi: 10.1093/molbev/msab199.

Wouter De Coster, Christine Van Broeckhoven, and Marc Cruts. Introducing nanoplot. *F1000Research*, 6:660, 2017. doi: 10.7490/f1000research.1114050.1.

Shifu Chen, Yanqing Zhou, Yaru Chen, and Jia Gu. fastp: an ultra-fast all-in-one fastq preprocessor. *Bioinformatics*, 34(17):i884–i890, 2018. doi: 10.1093/bioinformatics/bty560.

Aleksey V. Zimin. Masurca: Maryland super-reads celera assembler. https://github.com/alekseyzimin/masurca, 2025. Zugriff am: 2025-08-03.

Sergey Koren, Brian P. Walenz, Konstantin Berlin, Jason R. Miller, Nicholas H. Bergman, and Adam M. Phillippy. Canu: scalable and accurate long-read assembly via adaptive k-mer weighting and repeat separation. *Genome Research*, 27(5):722–736, 2017. doi: 10.1101/gr.215087.116.

Ryan R. Wick. Unicycler: Hybride assemblierung bakterieller genome. https://github.com/rrwick/Unicycler, 2025. Zugriff am: 2025-08-03.

Anton Bankevich, Sergey Nurk, Dmitry Antipov, Alexey A. Gurevich, Mikhail Dvorkin, Igor V. Kulikov, Konstantin M. Lesin, Sergey Nikolenko, Dmitry Pham, and Alexander P. Prjibelski. Spades: St. petersburg genome assembler. https://github.com/ablab/spades, 2025. Accessed: 2025-08-03.

Heng Li and Richard Durbin. Fast and accurate short read alignment with burrows-wheeler transform. *Bioinformatics*, 25(14):1754–1760, 2009. doi: 10.1093/bioinformatics/btp324. URL https://doi.org/10.1093/bioinformatics/btp324.

Abbildungsverzeichnis

1 Vergleich dreier Flye/Pilon-Assemblierungen auf Basis von ONT-Daten bei unterschiedlicher Abdeckung, visualisiert mit progressiveMauve 26

2 Vergleich von ONT-Assemblierungen mit unterschiedlichen Korrekturmethoden: (1) aus Publikation, (2) ohne Polierung, (3) mit Polierung und Nutzung gefilterter ONT-Daten, (4) mit Polierung und Nutzung von ONT-Rohdaten. 27

3 Übersichtliche Zusammenfassung aller in den vorherigen Abbildungen dargestellten Assemblierungen 28

4 Proteinsequenz des Pseudogens TALE1 (Assemblierung 3.6), wie sie im Alignment mit der funktionalen Variante aus Assemblierung 3.18 erscheint. 38

5 Proteinsequenz des Pseudogens TALE5 (Assemblierung 3.6), wie sie im Alignment mit der funktionalen Variante aus Assemblierung 3.18 erscheint. 39

6 Proteinsequenz des Pseudogens TALE18 (Assemblierung 3.6), wie sie im Alignment mit der funktionalen Variante aus Assemblierung 3.18 erscheint. 40

Tabellenverzeichnis

1 Auswirkung verschiedener `Filtlong`-Strategien auf ONT-Read-Qualität 8
2 Read-Qualität bei Illumina 8
3 Vergleich der Ergebnisse mit verschiedenen Coverage, Busco nach xanthomonadales . . . 11
4 Vergleich der Ergebnisse in Bezug auf die Anzahl der Polierungsiterationen bei Flye und Racon 12
5 Vergleich der Auswirkungen unterschiedlicher Anzahl von Flye-Polishing-Iterationen . . 15
6 Vergleich der Auswirkungen unterschiedlicher Anzahl von Flye-Polishing-Iterationen . . 15
7 Vergleich der Assemblierung mit Raven (3.12) und Canu (3.13) sowie Einfluss von Roh- versus gefilterten Daten auf das Polishing1 19
8 Vergleich der Assemblierung mit Raven (3.12) und Canu (3.13) sowie Einfluss von Roh- versus gefilterten Daten auf das Polishing2 20
9 Ergebnisse der Unicycler-Hybrid-Assemblierung 21
10 BUSCO-Ergebnisse für *xan* und *oryzae* vor und nach Polishing 22
11 Übersicht der Assemblierungs- und Polierschritte mit BUSCO-Ergebnissen (Xanthomonadales) 25
12 Vergleich von N50-Werten, TALE-Annotationen und Pseudogenen 25

Abkürzungsverzeichnis

DNA Desoxyribonukleinsäure

ENA European Nucleotide Archive

HMM Hidden-Markov-Modell

Indel Insertion oder Deletion von Nukleotiden in einer DNA-Sequenz

k-Mer Teilsequenz mit fester Länge k innerhalb einer DNA- oder RNA-Sequenz

NGS Next-Generation Sequencing

ONT Oxford Nanopore Technologies

PacBio Pacific Biosciences

RVD Repeat Variable Di-Residue

TALE Transcription Activator-Like Effector

TAL-Effektoren Transcription Activator-Like Effector

TALome Gesamtheit aller TALE-Gene eines Xanthomonas-Stamms

A Alignments der TALE-Proteinsequenzen

```
>tempTALE1_Pseudo
M-----------------------------------------------------------
-----------------------------------------AQCTD--------------
-----------------------------------------------------------
-----------------------------------------------------------
-------------------------------------------GCPLNLTPDQVVAIASN
IGGNQALETVQRLLPVLCQAHGLTPDQVVAIASHGGGKQALETVQRLLPVLCQDHGLTPD
QVVAIASNIGGKQALETVQRLLPVLCQDHGLTPDQVVAIASNIGGKQALETVQRLLPVLC
QDHGLTPDQVVAIASNIGGKQALETVQRLLPVLCQDHGLTPDQVVAIANNNGGKQALETV
QRLLPVLCQDHGLTPDQVVAIASHDGGKQALETVQRLLPVLCQGHGLTPDKVVAIASNSG
GKQALETVQRLLPVLCQAHGLTPDKVVAIANNNGGKQALETVQRLLPVLCQDHGLTPDQV
VAIASNSGGKQALETVQRLLPVLCQDHGLTPAQVVAIANNNGGKQALETVRRLLPVLCQD
HGLTPDQVVAIASHDGGKQALETVQRLLPVLCQDHGLTPDQVVAIANNNGGKQALETVQR
LLPVLCQDHGLTQDQVVAIASHIGGKQALETVQRLLPVLCQDHGLTPDQVVAIASHDGGK
QALETVQRLLPVLCQDHGLTPAQVVAIANNNGGKQALETVQRLLPVLCQDHGLSPDQVVA
IASNIGGKQALETVQRLLPVLCQDHGLTPDQVVAIASNGGGKQALETVQRLLPVLCQDHG
LSPDQVVAIASHDGGKQALETVQRLLPVLCQDHGLTPDQVVAIASNGGGKQALESIVAQL
SRPDPALAALTNDHLVALACLGGRPAMDAVKKGLPHAPELIRRVNSRIGERTSHRVADYA
QVVRVLEFFQCHSHPAYAFDEAMTQFGMSRNGLVQLFRRVGVTELEARCGTLP-------
-----------QP--------------------RSVGTVSSRHQG*-------------
-----------------------------------------------------------
---------------------------------------
```

Abbildung 4: Proteinsequenz des Pseudogens TALE1 (Assemblierung 3.6), wie sie im Alignment mit der funktionalen Variante aus Assemblierung 3.18 erscheint.

```
>tempTALE5_Pseudo
MDPIRSR---TPSP-------------------------------------------A
RELLPG---PQPDRVQPTADRGGGSA--CWRPLDGLPARRTMSRTRLPSPPAPSPAFPAG
SFSDLLRQVDTSLIDSMPAVGTPHTAAAPAEWDEMQSGLRAADDPPPTVRVAVTAARPPR
AKPAPRRRAAQPSDASPAAQVDLRTLGYSQQQQEKIKPKVRSTVAQHHEALVGHGFTHAH
IVALSQHPAALGTVAVTYQDIITALPEATHEDIVGVGKQWSGARALKALLTEAGELRGPP
LQLDTGQLLKIAKRGGVTAVQAVHAWRNALTGAPLNLTPDQVVAIASNSGGKQALETVQR
LLPVLCQDHGLTPAQVVAIASHDGGKQALETVQRLLPVLCQDHGLTLDKVVAIASNGGKQ
ALETVQRLLPVLCQDHGLTPNQVVAIASNSGGKQALETVQRLLPVLCQDHGLTPNQVVAI
ASNGGKQALESIVAQLSRPDPALAALTNDHLVALACLGGRPALDAVK------------
------------------------------------------------------------
------------------------------------------------------------
-----------------------------------RDC------------RTR-------
-----------------------------------------------------RN*
```

Abbildung 5: Proteinsequenz des Pseudogens TALE5 (Assemblierung 3.6), wie sie im Alignment mit der funktionalen Variante aus Assemblierung 3.18 erscheint.

```
>tempTALE18_Pseudo
MR-VRDRLHVCASPWSSRVARITRSCVLPCGLGSYVGTTDR*SGGDHVKRYA*WIPFVRA
RQVLPASFCPDPNRIGFSRLQIGGSACWRPLDGLPARRTMSRTRLPSPPAPSPAFSAGSF
SDPLRQFDPSLLDTSLFDSMPAVGTPHTAAAPAEWDEAQSALRAADDPPPTVRVAVTAAR
PPRAKPAPRRRAAQPSDASPAAQVDLRTLGYSQQQQEKIKPKVRSTVAQHHEALVGHGFT
HAHIVALSKHPAALGTVAVTYQHIITALPEATHEDIVGVGKQWSGARALEALLTDAGELR
GPPLQLDTGQLLKIAKRGGVTAVEAVHASRNALTGAPLNLTPAQVVAIASNNGGKQALET
VQRLLPVLCQAHGLTPAQVVAIASHDGGKQALETVQRLLPVLCQDHGLTPDQVVAIASNS
GGKQALETVQRLLPVLCQDHGLTPDQVVAIASNGGGKQALETVQRLLPVLCQDHGLTPDQ
VVAIASHDGGKQALETVQRLLPVLCQDHGLTPDQLVAIANNNGGKQALETVQRLLPVLCQ
DHGLTPDQVVAIASNGGKQALETVQRLLPVLCQDHGLTPDQVVAIASNIGGKQALETVQR
LLPVLCQAHGLTPAQVVAIASHDGGKQALETMQRLLPVLCQAHGLTPAQVVAIASNSGGK
QALETVQRLLPVLCQAHGLTPAQVVAIVSHDGGKQALETVQRLLPVLCQAHGLTPDQVVA
IASNNGGKQALETVQRLLPVLCQDHGLTPDQVVAIASHDGGKQALGTVQRLLPVLCQDHG
LTPDQVVAIASNNGGKQALETVQRLLPVLCQDHGLTPDQVVAIASHDGGKQALETVQRLL
PVLCQAHGLTPAQVVAIASNNGGKQALETVQRLLPVLCQDHGLTPDQVVAIANNNGGKQA
LETVQRLLPVLCQAHGLTPAQVVAIASNNGSKQALETVQRLLPVLCQAHGLTPAQVVAIA
SNNGGKQALETVQRLLPVLCQAHGLTPDQVVAIASNNGGKQALETVQRLLPVLCQAHGLT
PDQVVAIASNNGGKQALETVQRLLPVLCQAHGLTPDQVVAIASNNGGKQALETVQRLLPV
LCQDHGLTPAQVVAIASHDGGKQALETVQRLLPVLCQDHGLTPAQVVAIASNGGGKQALE
SIVAQLSRPDPALAALTNDHLVALACLGGRPALDAVKK----------------------
------------------------------------------------------------
------------------DC--------------------------------------
-----------------------------------------RTR-----------------
-------------------------------------------RN*
```

Abbildung 6: Proteinsequenz des Pseudogens TALE18 (Assemblierung 3.6), wie sie im Alignment mit der funktionalen Variante aus Assemblierung 3.18 erscheint.

B T-Coffee Alignment-Ergebnis

```
T-COFFEE, Version_13.46.0.919e8c6b (2023-07-07 22:06:42 - Revision 29996c5 - Build 980)
Cedric Notredame
CPU TIME:0 sec.
SCORE=1000
* .
  BAD AVG GOOD
*
tempTALES       : 100
tempTALES_Pseud : 100
cons            : 100

tempTALES       ATGCGGGTCCGGGATCGCCTTCATGTCTGCGCCTCACCCTGGTCG
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       TCGAGGGTTGCCAGGATCACCCGAAGTTGTGTACTGCCATGCGGC
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       CTCGGAAGCTATGTAGGGACCACAGACCGCTAGTCTGGAGGCGAC
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       CATGTAAAGAGGTATGCCTGATGGATCCCATTCGTTCGCGTACGC
tempTALES_Pseud --------------------ATGGATCCCATTCGTTCGCGTACGC

cons                                 **************************

tempTALES       CAAGTCCTGCCCGCGAGCTTCTGCCCGGACCCCAACCGGATAGGG
tempTALES_Pseud CAAGTCCTGCCCGCGAGCTTCTGCCCGGACCCCAACCGGATAGGG

cons            **************************************************

tempTALES       TTCAGCCGACTGCAGATCGGGGGGGGGGGCTCCGCCTGCTGGCGGC
tempTALES_Pseud TTCAGCCGACTGCAGATCGGGGGGGGGGCTCCGCCTGCTGGCGG-

cons            *************************************************

tempTALES       CCCCTGGATGGCTTGCCCGCTCGGCGGACGATGTCCCCGGACCCGG
tempTALES_Pseud CCCCTGGATGGCTTGCCCGCTCGGCGGACGATGTCCCGGACCCGG

cons            **************************************************

tempTALES       CTGCCATCTCCCCCTGCGCCCTCGCCTGCGTTCCCGGCGGGCAGC
tempTALES_Pseud CTGCCATCTCCCCCTGCGCCCTCGCCTGCGTTCCCGGCGGGCAGC

cons            **************************************************

tempTALES       TTCAGCGATCTGCTCCGTCAAGTCGATACATCGCTTATTGATTCG
tempTALES_Pseud TTCAGCGATCTGCTCCGTCAAGTCGATACATCGCTTATTGATTCG

cons            **************************************************

tempTALES       ATGCCTGCCGTCGGCACGCCGCATACAGCGGCTGCCCCAGCAGAG
tempTALES_Pseud ATGCCTGCCGTCGGCACGCCGCATACAGCGGCTGCCCCAGCAGAG

cons            **************************************************

tempTALES       TGGGATGAGATGCAATCGGGTCTGCGTGCAGCCGATGACCCGCCA
tempTALES_Pseud TGGGATGAGATGCAATCGGGTCTGCGTGCAGCCGATGACCCGCCA

cons            **************************************************

tempTALES       CCCACCGTGCGTGTCCGCTGTCACTGCCGCCGCCGGCCGCCGCGCC
tempTALES_Pseud CCCACCGTGCGTGTCCGCTGTCACTGCCGCCGCCGGCCGCCGCGCCC

cons            **************************************************

tempTALES       AAGCCGGCCCCGCCGACGGCGTGCGGCGCAACCCTCCGACGCTTCG
tempTALES_Pseud AAGCCGGCCCCGCCGACGGCGTGCGGCGCAACCCTCCGACGCTTCG

cons            **************************************************

tempTALES       CCGGCCGCCGCAGGTGGATCTACGCCACGCTCGGCTACAGTCAGCAG
tempTALES_Pseud CCGGCCGCCGCAGGTGGATCTACGCCACGCTCGGCTACAGTCAGCAG

cons            **************************************************

tempTALES       CAGCAAGAAGAAGATCAAACCGAAGGTGCGTTCGACAGTGGCGCAG
tempTALES_Pseud CAGCAAGAAGAAGATCAAACCGAAGGTGCGTTCGACAGTGGCGCAG

cons            **************************************************

tempTALES       CACCACGAGGCACTGGTGGGCCATGGGTTTACACACGCGCACATC
tempTALES_Pseud CACCACGAGGCACTGGTGGGCCATGGGTTTACACACGCGCACATC

cons            **************************************************

tempTALES       GTTGCGCTCAGCCAACACCCGGCAGCGTTAGGGACCGTTGCTGTC
tempTALES_Pseud GTTGCGCTCAGCCAACACCCGGCAGCGTTAGGGACCGTTGCTGTC

cons            **************************************************

tempTALES       ACGTATCAGGACATAATCACGGCGTTGCCAGAGGCGACACACGAA
tempTALES_Pseud ACGTATCAGGACATAATCACGGCGTTGCCAGAGGCGACACACGAA

cons            **************************************************

tempTALES       GACATCGTTGGCGTCGGCCAAACAGTGGTCCGGCGCACGCGCCCTG
tempTALES_Pseud GACATCGTTGGCGTCGGCAAACAGTGGTCCGGCGCACGCGCCCTG

cons            **************************************************

tempTALES       AAGGCCTTGCTCACGGAGGCGGGGGAGTTGAGAGGTCCGCCGTTA
tempTALES_Pseud AAGGCCTTGCTCACGGAGGCGGGGGAGTTGAGAGGTCCGCCGTTA

cons            **************************************************

tempTALES       CAGTTGGACACAGGCCAACTTCTCAAGATTGCAAAACGTGGCGGC
tempTALES_Pseud CAGTTGGACACAGGCCAACTTCTCAAGATTGCAAAACGTGGCGGC

cons            **************************************************

tempTALES       GTGACCGCAGTGCAGGCAGTGCATGCATGGCGCAATGCACTGACG
tempTALES_Pseud GTGACCGCAGTGCAGGCAGTGCATGCATGGCGCAATGCACTGACG

cons            **************************************************

tempTALES       GGTGCCCCCCTGAACCTGACCCCGGACCAAGTGGTGGCCATCGCC
tempTALES_Pseud GGTGCCCCCCTGAACCTGACCCCGGACCAAGTGGTGGCCATCGCC

cons            **************************************************

tempTALES       AGCAATAGTGGCGGCAAACAGGCGCTGGAGACGGTGCAGCGGCTG
tempTALES_Pseud AGCAATAGTGGCGGCAAACAGGCGCTGGAGACGGTGCAGCGGCTG

cons            **************************************************

tempTALES       TTGCCGGTGCTGTGCCAGGACCATGGCCTGACCCCGGCCCAGGTG
tempTALES_Pseud TTGCCGGTGCTGTGCCAGGACCATGGCCTGACCCCGGCCCAGGTG

cons            **************************************************

tempTALES       GTGGCCATCGCCAGCCACGATGGCGGCAAGCAGGCGCTGGAGACG
tempTALES_Pseud GTGGCCATCGCCAGCCACGATGGCGGCAAGCAGGCGCTGGAGACG

cons            **************************************************

tempTALES       GTGCAGCGGCTGTTGCCGGTGCTGTGCCAGGACCATGGCCTGACC
tempTALES_Pseud GTGCAGCGGCTGTTGCCGGTGCTGTGCCAGGACCATGGCCTGACC

cons            **************************************************

tempTALES       CTGGACAAGGTGGTGGCCATCGCCCAGCAATGGCGGCAAGCAGGCG
tempTALES_Pseud CTGGACAAGGTGGTGGCCATCGCCAGCAATGGCGGCAAGCAGGCG

cons            **************************************************

tempTALES       CTGGAGACGGTGCAGCGGCTGTTGCCGGTGCTGTGCCAGGACCAT
tempTALES_Pseud CTGGAGACGGTGCAGCGGCTGTTGCCGGTGCTGTGCCAGGACCAT

cons            **************************************************

tempTALES       GGCCTGACCCCGAACCAGGTGGTGGCCATCGCCAGCAATAGTGGC
tempTALES_Pseud GGCCTGACCCCGAACCAGGTGGTGGCCATCGCCAGCAATAGTGGC

cons            **************************************************

tempTALES       GGCAAGCAGGCGCTGGAGACGGTGCAGCGGCTGTTGCCGGTGCTG
tempTALES_Pseud GGCAAGCAGGCGCTGGAGACGGTGCAGCGGCTGTTGCCGGTGCTG

cons            **************************************************

tempTALES       TGCCAGGACCATGGCCTGACCCCGAACCAGGTGGTGGCCATCGCC
tempTALES_Pseud TGCCAGGACCATGGCCTGACCCCGAACCAGGTGGTGGCCATCGCC

cons            **************************************************

tempTALES       AGCAATGGCGGCAAGCAGGCGCTGGAGAGCATTGTTGCCCAGTTA
tempTALES_Pseud AGCAATGGCGGCAAGCAGGCGCTGGAGAGCATTGTTGCCCAGTTA

cons            **************************************************

tempTALES       TCTCGCCCTGATCCGGCGTTGGCCGCGTTGACCAACGACCACCTC
tempTALES_Pseud TCTCGCCCTGATCCGGCGTTGGCCGCGTTGACCAACGACCACCTC

cons            **************************************************

tempTALES       GTCGCCTTGGCCTGCCTCGGCGGACGTCCTGCCCTGGATGCAGTG
tempTALES_Pseud GTCGCCTTGGCCTGCCTCGGCGGACGTCCTGCCCTGGATGCAGTG

cons            **************************************************

tempTALES       AAAAAGGGATTGCCGCACGCGCCGGAATTGATCAGAAGGAATCAAT
tempTALES_Pseud -AAAAGGGATTGCCGCACGCGCCGGAATTGA----------------

cons            ***************************************

tempTALES       CGCCGCATTCCCGAACGCACGTCCATCGCGTTCCCGACCTCGCG
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       CACGTGGTTCGCGTGCTTGGTTTTTTCCAGAGCCACTCCCACCCA
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       GCGCAAGCATTCGATGACGCCATGACGCAGTTCGAGATGAGCAGG
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       CACGGCTTGGTACAGCTCTTTCGCAGAGTGGGCGTCACCGAATTC
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       GAAGCCCGCTACGGAACGCTCCCCCCAGCCTCGCAGCGTTGGGAC
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       CGTATCCTCCAGGCATCAGGGATGAAAAGGGCCAAACCGTCCCCT
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       ACTTCAGCTCAAACACCGGATCAGGCGTCTTTGCATGCATTCGCC
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       GATTCGCTGGAGCGTGACCTTGATCGCGCCTAGCCCAATGCACGAG
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       GGAGATCAGACAGGGGCAAGCAGCCGTAAACGGTCCCGATCGGAT
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       CGTGCTGTCACCGGCCCCTCCGCACAGCAATCTTTCGAGGTGCGC
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       GTTCCCGAACAGCGCGATGCGCTGCATTTGCCCCTCAGCTGGAGG
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       GTAAAACGCCCGGCGTACCAGGATCGGGGGCGGCCTCCCGGATCCT
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       GGTACGCCCATCGCTGCCGACCTGGCAGCGTCCAGCACCGTGATG
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       TGGGAACAAGATGCGGCCCCCTTCGCAGGGGCAGCGGATGATTTC
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       CCGGCATTCAACGAAGAGGAGCTCGCATGGTTGATGGAGCTATTG
tempTALES_Pseud ----------------------------------------------

cons            

tempTALES       CCTCAGTCAGGCTCAGTCGGAGGGACGATC
tempTALES_Pseud -------------------------------

cons            
```